QCon
全球软件开发大会

InfoQ 技术大会

贯穿系统栈的强化学习加速

历史 Rollout复用、协作式 GPU Kernel与高效 Agent Sandbox

和敬凯、杜冬冬（Dong Du）
上海交通大学



合作研究：Guangda Sun、Tianjian Li、Yunpeng Dong, Erhu Feng、Yubin Xia、Haibo Chen

讲者介绍



和敬凯

和敬凯：上海交通大学并行与分布式系统实验博士生

研究方向： 操作系统，AI基础设施

研究成果： 发表在SOSP、OSDI、ASPLOS等国际著名会议和期刊（以第一作者身份）。**获得 SOSP 2025 最佳论文奖（近60年来亚洲第二次）。**



杜冬冬（Dong Du）

杜冬冬：上海交通大学并行与分布式系统实验室 / 上交-华为先进研究中心 副教授

研究方向： 人工智能操作系统

研究成果： 发表在SOSP、OSDI、ASPLOS、ISCA等国际著名会议和期刊。**获得 SOSP 2025 最佳论文奖， FAST 2026 最佳论文奖和突出技术奖。**

成果应用： 鸿蒙D-VSync 渲染、字节强化学习框架VeRL 投机解码、TileLang算子框架、Linux CRIU并行 GPU启动等

指令集架构： SPMP 安全扩展合入 RISC-V 指令集架构，中国目前唯一合入的大颗粒扩展（人民日报等报道）

后训练时代：RL 已成为主流实践

强化学习（RL）

→ 基础模型对齐的主流方法

→ 将基础模型转化为智能 Agent

→ 模型：ChatGPT O1、DeepSeek R1、Gemini 3 等

→ 算法

- → 2024 年及以前：PPO
- → 2025.2：GRPO（DeepSeek R1）
- → 2025.5：DAPO（ByteDance Seed）
- → 2025.7：GSPO（Qwen-3）
- → …

deepseek

OpenAI Gemini

Claude Grok



标准 RL 训练流程（GRPO / DAPO / GSPO / …）

1. Rollout（生成）

以自回归方式生成响应。

长序列与长度差异主导 Rollout 开销。

2. 评估（奖励）

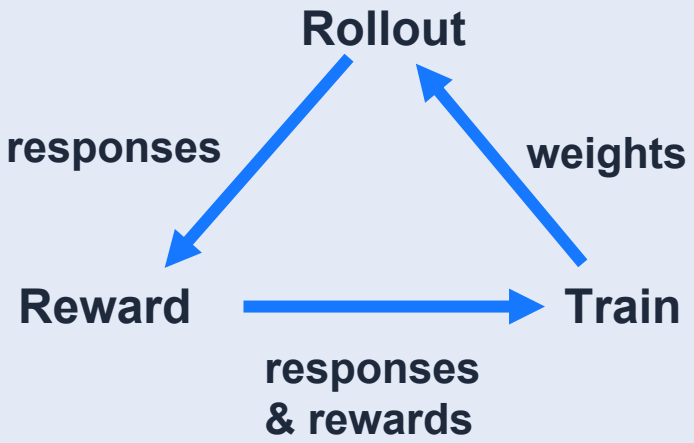
通过奖励模型或验证器评估响应。

Agent 任务还需在有状态环境中执行工具。

3. 优化（训练）

通过反向传播更新 Actor 模型。

GPU Kernel 也支撑 Rollout 与奖励计算。



RL 与 Agent 训练的三类开销

1 / Rollout 生成

自回归序列长、响应长度不均，
使 GPU 产生等待。

2 / GPU Kernel 执行

动态任务与跨 SM 依赖造成负载不均、
同步停顿，以及反复的 HBM 访存。

3 / Agent 环境执行

分支探索需要反复保存、恢复和分叉
文件、内存、进程与连接等状态。

三项工作：复用不同层次的已有信息

RhymeRL / 大模型 RL 训练加速系统

复用历史回答，加快 Rollout 生成。
根据历史长度调度任务，减少 GPU cluster 中的等待。

Meld / GPU Kernel 编译与执行框架

面向动态输入，减少跨 SM 同步和重复访存。
根据计算与数据依赖，安排细粒度跨 SM 协作。

DeltaBox / 支持快速状态切换的 Agent Sandbox

面向 Agent 分支探索，快速保存、回滚和分叉环境。
只保存变化的文件与进程状态，复用未变化的部分。

01

RhymeRL：大模型 RL 训练加速系统

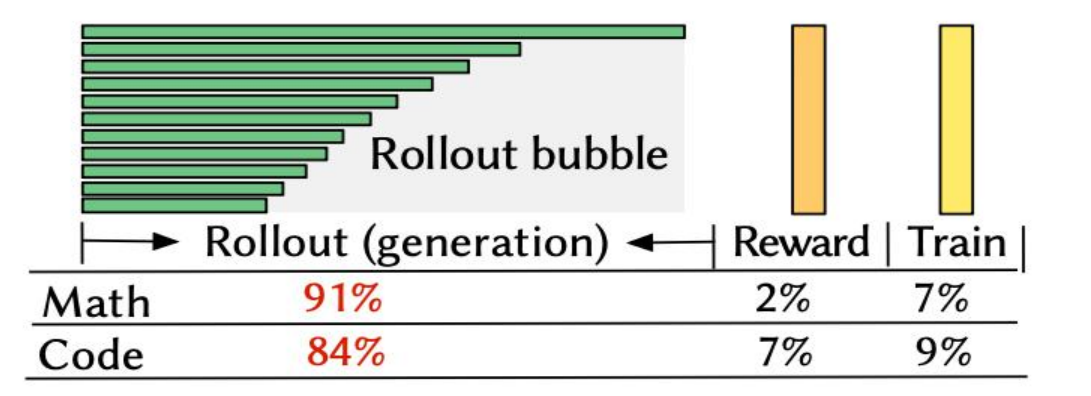
Rollout 生成慢，GPU cluster 中存在长尾等待。

利用历史回答加速生成，并按历史长度调度任务。

Rollout 是 RL 训练的主要瓶颈

结论 #1：Rollout 占据 RL 训练的大部分时间

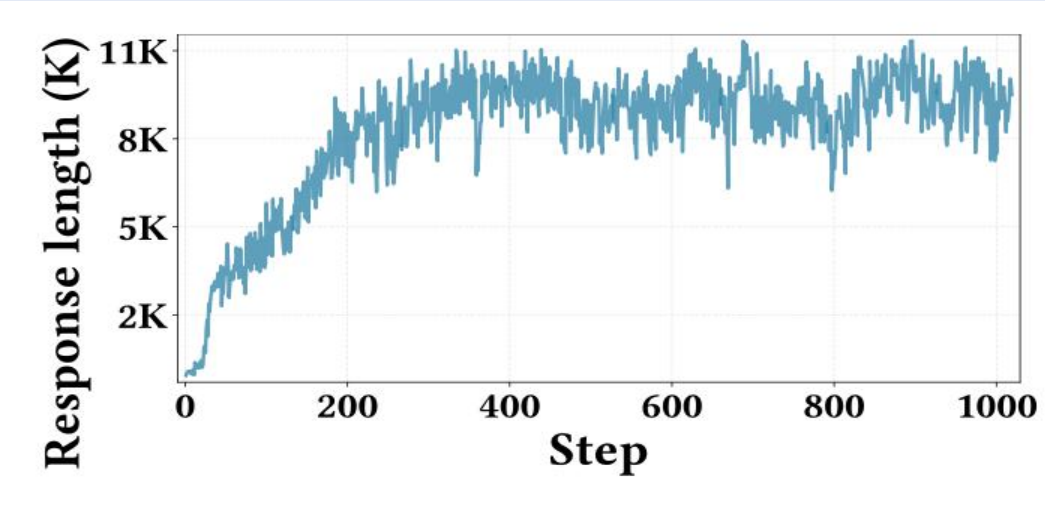
→ 在 RLVR 训练（数学 / 代码）中占 84%–91%



LLM RL 各阶段及其时间占比

原因

1. 复杂推理产生长响应，且响应长度随训练增长
2. 解码受内存带宽限制，计算密度低



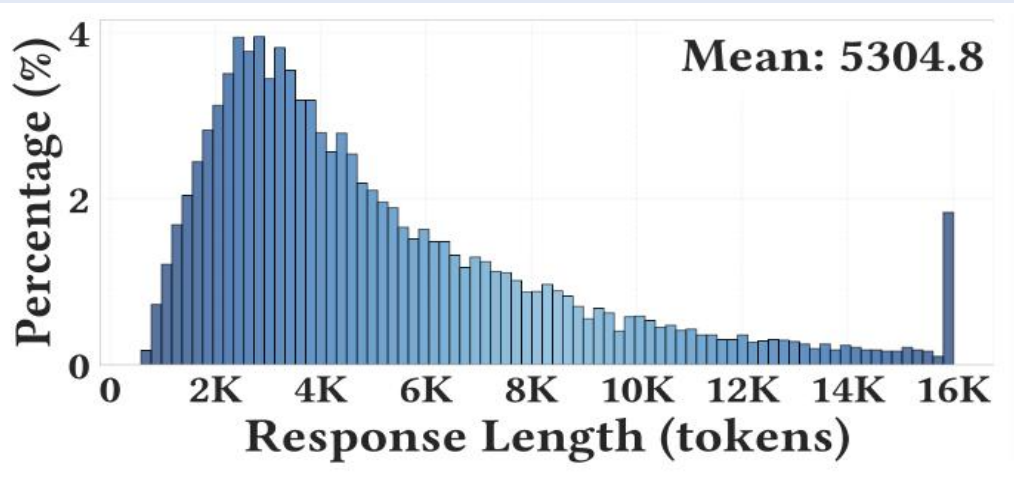
不同训练步的平均响应长度

Rollout 是 RL 训练的主要瓶颈

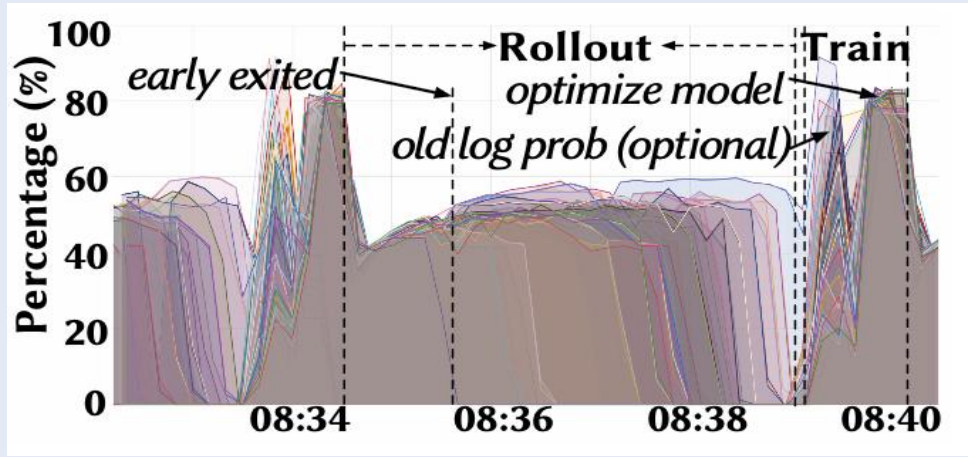
结论 #2：Rollout 负载不均导致 GPU 利用率不足

→ 同一 batch 内存在显著长尾

→ 最早完成的 GPU 在整个 Rollout 期间约有 76% 的时间空闲



单个训练步中的响应长度分布



实际 GPU SM 利用率

RL 中的推理：反复迭代训练集

RL 训练（如 GRPO）是对训练集进行**迭代、重复处理**的过程

→ **重复推理**：现代 RL 算法采用组内相对优势，为每个 prompt 生成一组响应（16–64 条）

→ **迭代生成**：RL 训练跨越多个 epoch，反复将同一批 prompt 输入模型

→ **Qwen3 仅用 3995 对问题与验证器，训练 170 步**

→ **DeepSeekMath-RL（> 8000 步）训练 > 55 个 epoch**

→ **DeepScaleR（1750 步）训练 > 5 个 epoch**

关键洞察：RL Rollout 的历史相似性

我们观察到：不同 epoch 的 Rollout 响应具有**很强的历史相似性**

Token 序列相似性

同一 prompt 在不同 epoch 的响应

共享相似的 token 子序列。

响应长度分布相似性

按响应长度排序后，prompt 的排名

在相邻 epoch 间保持相似。

历史相似性 #1：Token 相似性

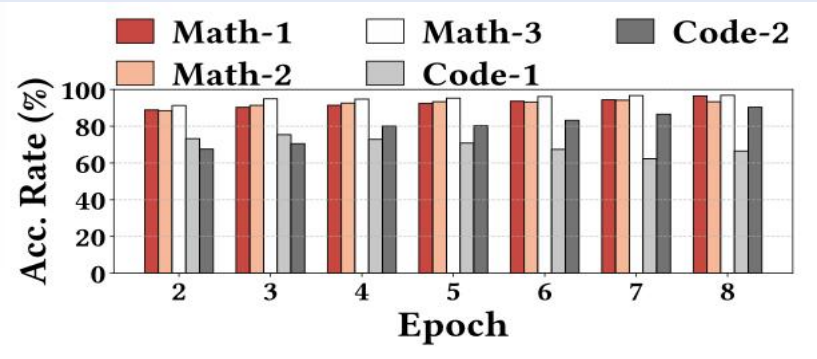
同一 prompt 的响应包含重叠的 token 子序列

量化分析

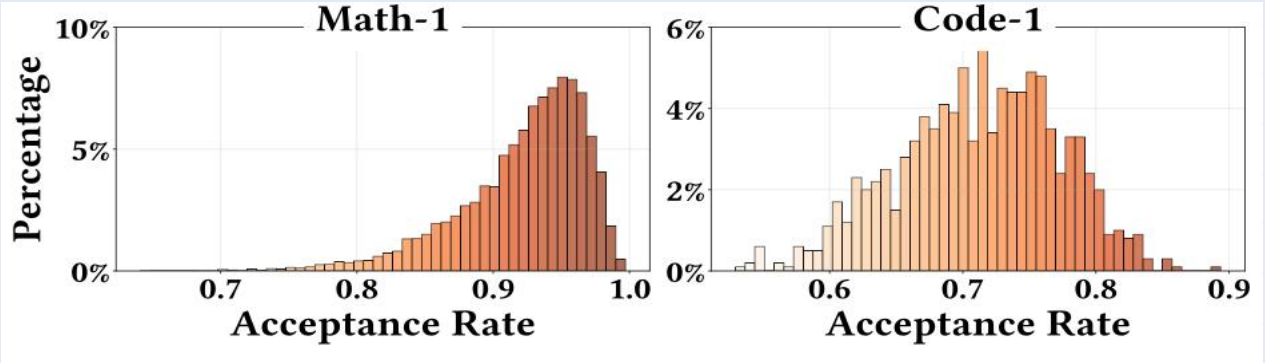
→ 数学 / 代码 RL 的真实轨迹，30 万余条数据

→ 基于前缀查找匹配子序列

→ 75%–93% 的 token 可匹配



不同 epoch 的接收率



响应接收率分布

历史相似性 #2：长度分布相似性

相邻 epoch 中，prompt 的响应长度排名相似

量化分析

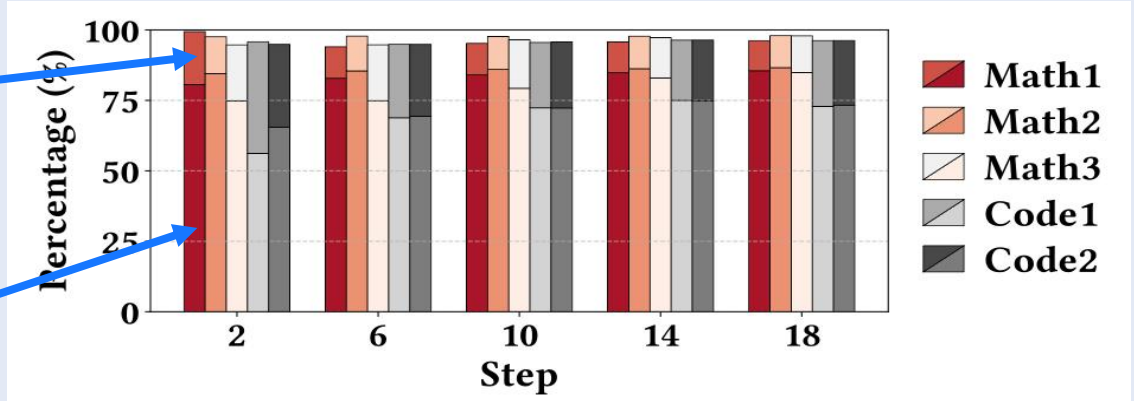
→ 20 个 epoch，8 个排名组

→ 按每个 prompt 所有响应的长度中位数排序

→ 仅 16%–18% 的 prompt 升至更高排名组

柱状图上部：仅升一组，且长度处于新组较短一半的 prompt

柱状图下部：留在原组或降至更低组的 prompt



响应在 20 个 epoch 中的排名组变化

RhymeRL 的两个主要机制

问题 #1

带宽受限的 Rollout

主导训练时间

+

洞察 #1

Token 序列相似性

→

HistoSpec

投机解码

利用历史响应

加速 Rollout 生成

问题 #2

Rollout 负载不均

导致 GPU 利用率不足

+

洞察 #2

响应长度分布相似性

→

HistoPipe

主动负载均衡

按历史响应长度

调度 Rollout 任务

HistoSpec：投机式 Rollout 生成

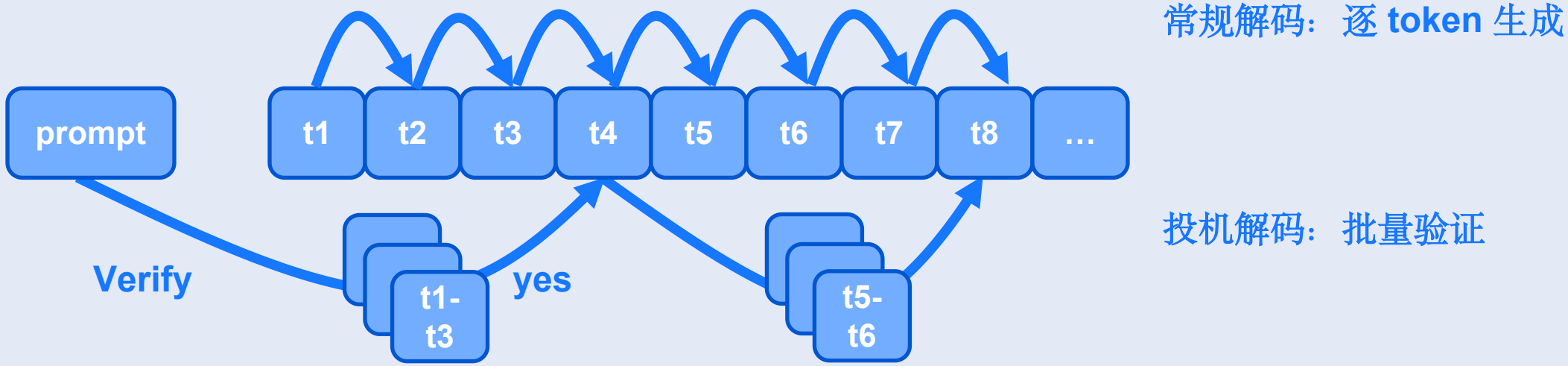
投机解码

→ 用轻量方法预测多个草稿 token

→ LLM 在一次前向计算中并行验证这些 token

→ LLM 接收有效 token

→ 理论上已证明可保持输出分布不变



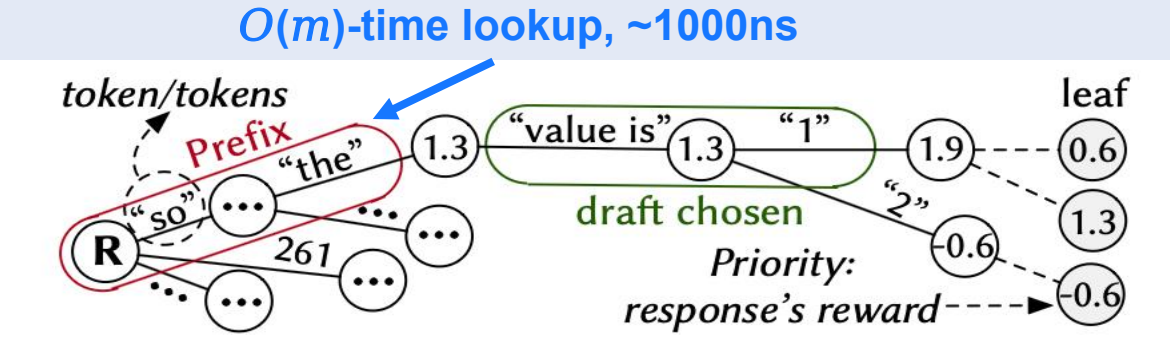
HistoSpec：历史索引与本地草稿检索

离线：History Worker 利用空闲 CPU 为历史响应建立索引。

在线：Rollout Worker 从各 prompt 的后缀树中检索草稿。

长度为 m 的前缀，查找时间复杂度为 O(m)。

按奖励加权的分支优先选择更有潜力的后续序列。



基于后缀树的草稿生成

HistoSpec：借鉴 AIMD 的 Token 投机

观察：匹配长度与出现概率不一致

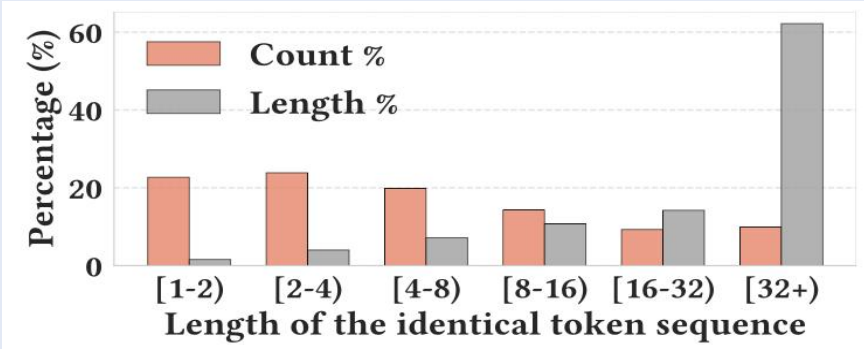
→ 短片段（1–2 个 token）数量多，长片段则贡献大部分总匹配长度

方案：借鉴 AIMD，动态调整草稿窗口

→ TCP 拥塞控制中的 AIMD：加性增大、乘性减小

→ RhymeRL：接收时加性增大，拒绝时重置为最小值

→ 长匹配序列保持高计算密度，短序列尽量减少无效计算



投机命中长度分布

HistoPipe：感知长度分布的混合流水线

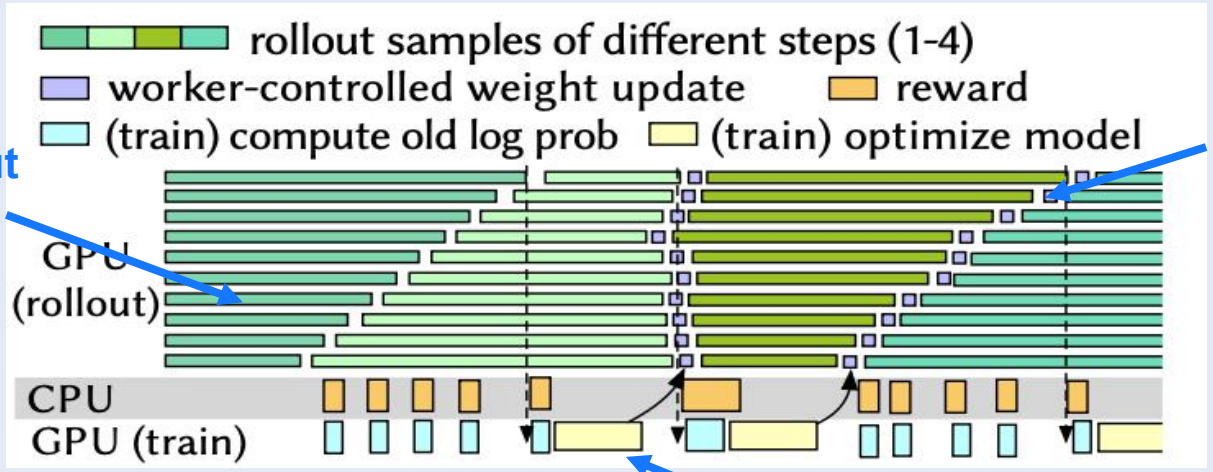
One-step-off 训练仍存在 Rollout 长尾气泡。

利用历史信息预测 prompt 的响应长度排名。

不同训练步交替采用长度升序与降序分组，

使 Rollout 与训练负载互补。

跨训练步互补的 **Rollout**



基于主机端权重缓冲区的

异步权重更新

One-step-off Rollout–Train 流水线

HistoPipe：基于迁移的长尾再均衡

挑战：异常长的 Rollout

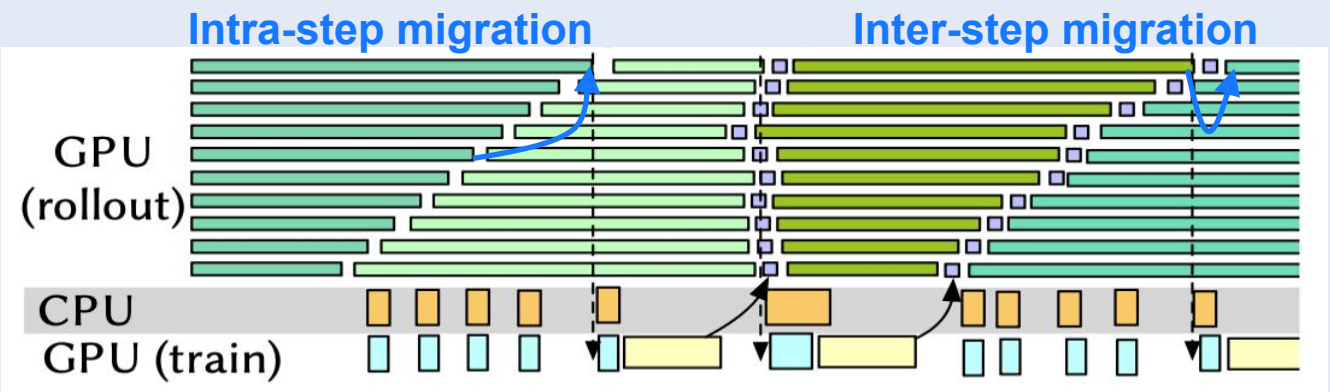
→ 破坏负载互补，打断流水线

→ 占比虽少，但仍有 2%–5%

方案：基于迁移的长尾再均衡

→ 步内迁移：将长 Rollout 迁至仍在执行本步 Rollout 的其他组

→ 跨步迁移：将异常长的 Rollout 迁移至下一训练步

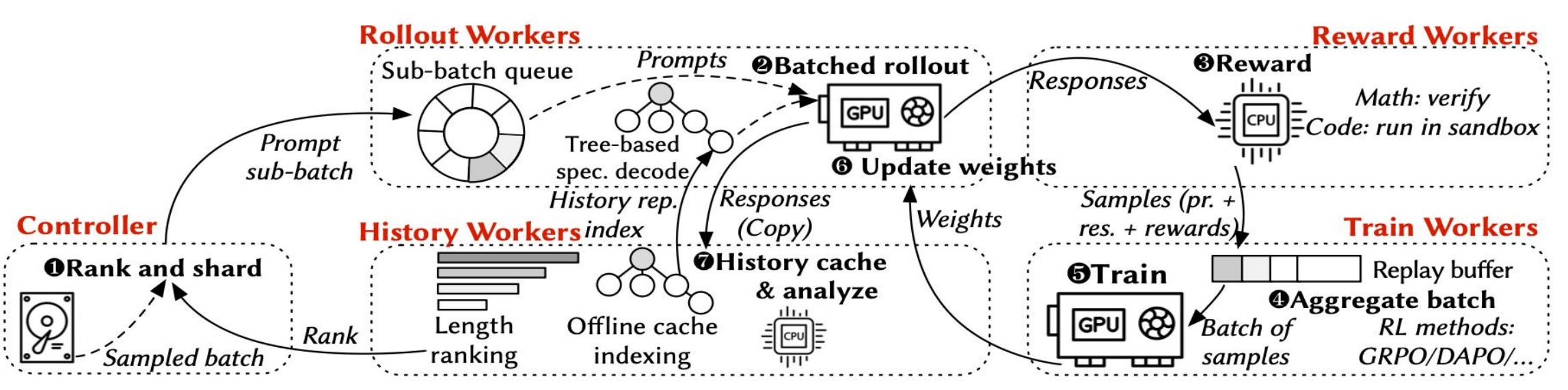


RhymeRL：生成与调度的协同

(1) HistoPipe：按长度安排负载

(2) HistoSpec：投机生成

(3) 计算奖励



(7) 更新历史缓存

(5–6) 更新模型并传输权重（异步）

(4) 聚合并打乱 batch

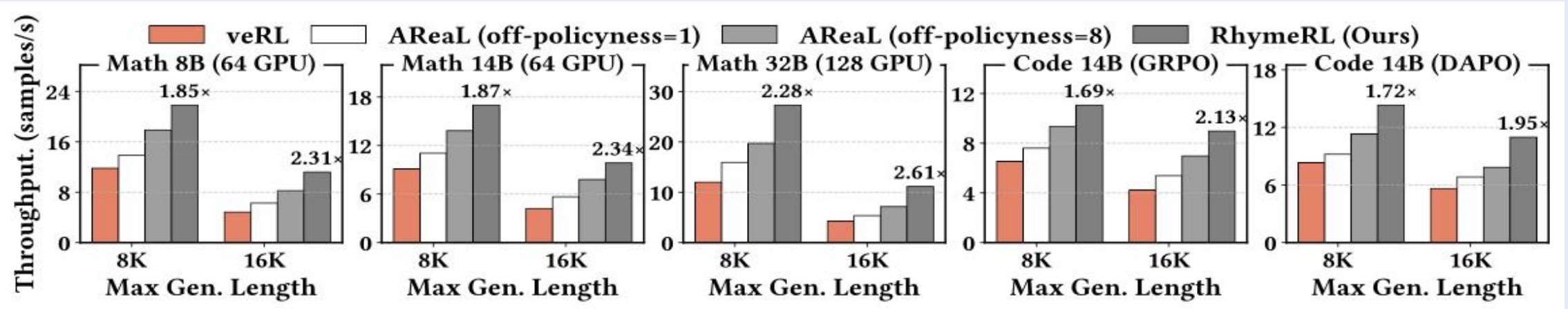
RhymeRL：端到端训练吞吐

64–128 张 Hopper GPU；GRPO / DAPO；数学与代码任务

相较 veRL（one-step-off），吞吐最高达 2.6×

相较 AReaL（off-policyness = 8），最高达 1.6×

实验覆盖 8K 和 16K 的最大生成长度。

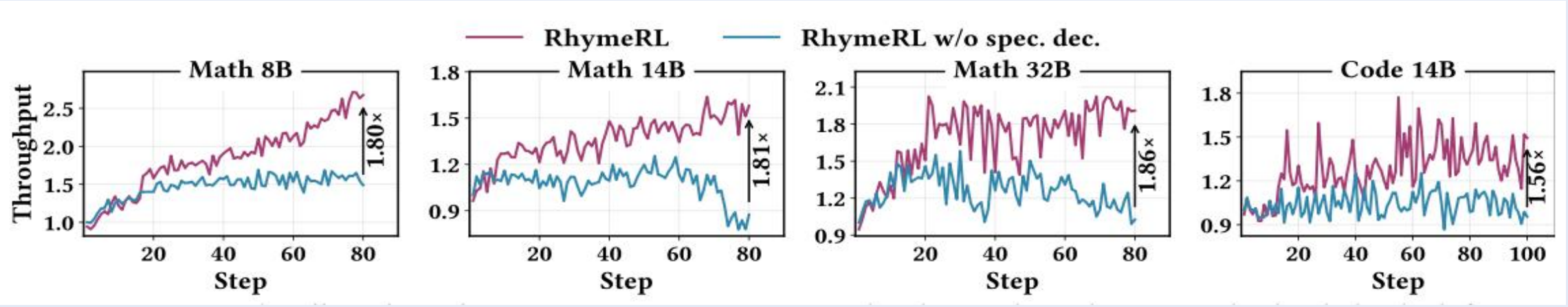


RhymeRL、veRL 与 AReaL 的训练吞吐

RhymeRL：投机解码与流水线的收益

HistoSpec：Rollout 吞吐最高达 1.86×

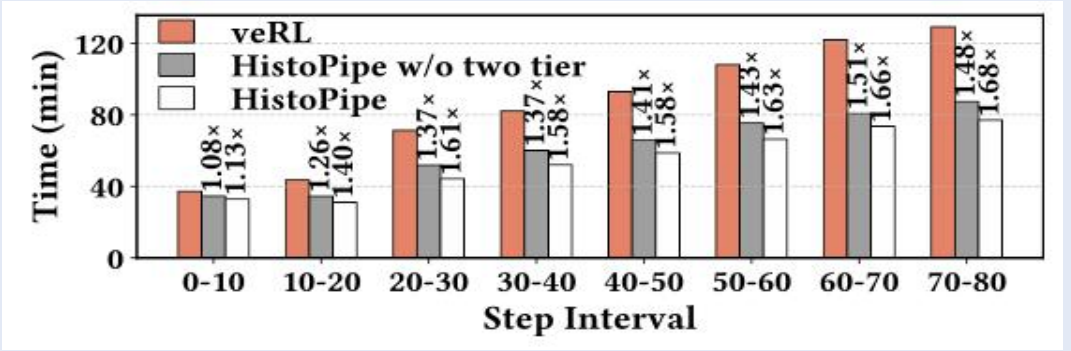
65%–79% 接收率



HistoPipe：最高加速 1.68×

每 10 个训练步的耗时

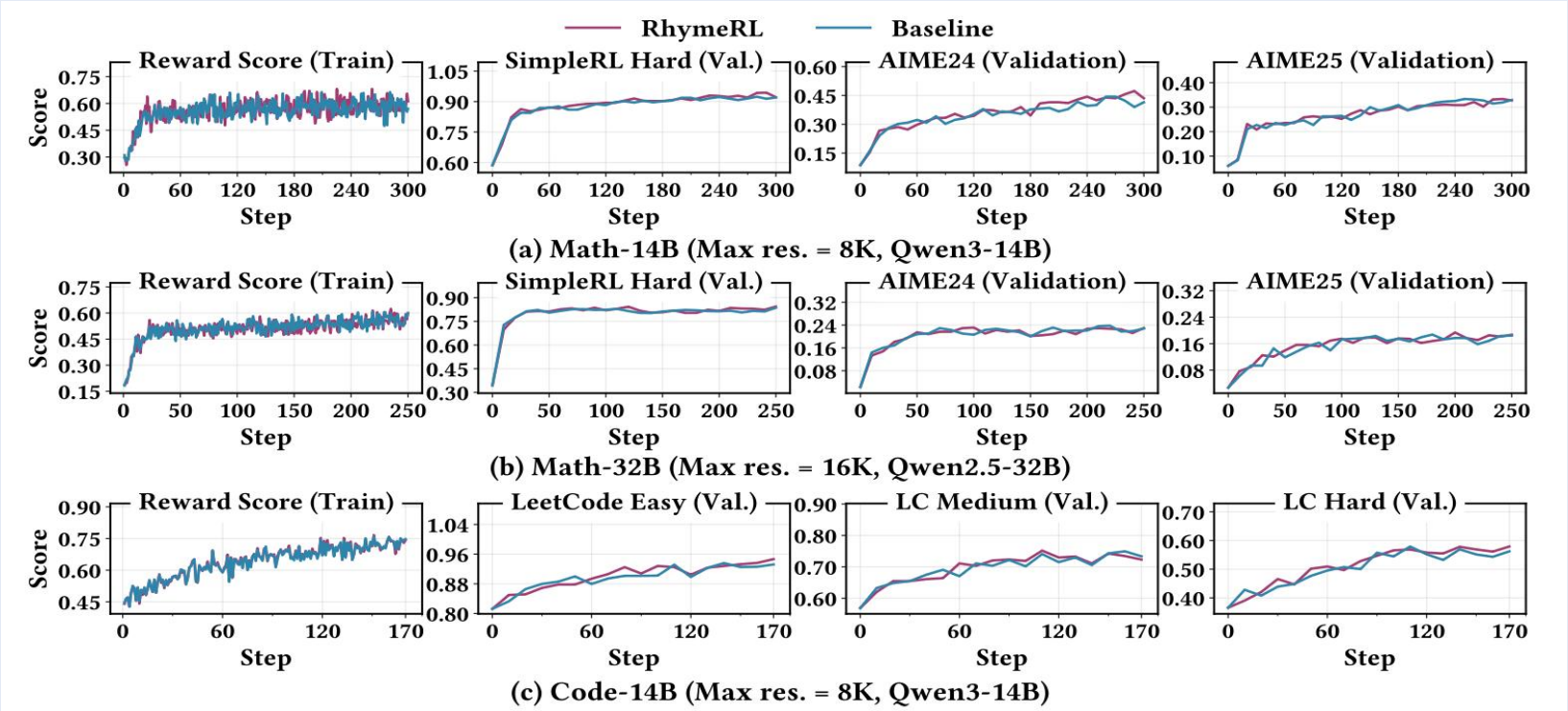
Two-tier Scheduling：相较朴素混合流水线
最高再加速 1.14×



RhymeRL：保持训练轨迹与模型效果一致

→ RhymeRL 的训练轨迹与 veRL 一致

→ 训练所得模型的端到端效果与 veRL 一致



端到端训练曲线与模型验证分数

02

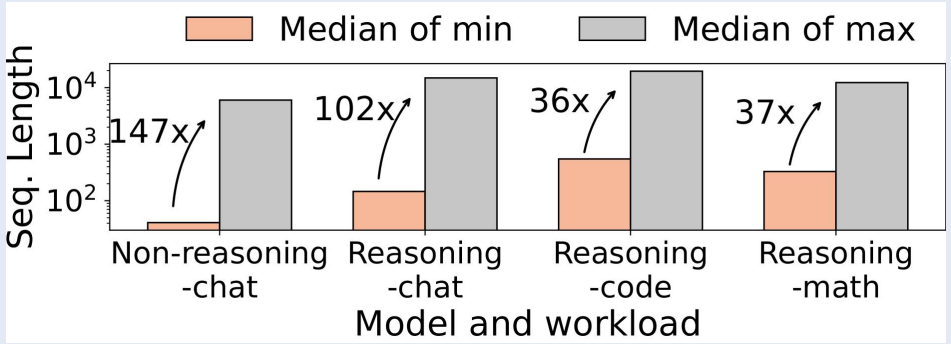
Meld：GPU Kernel 编译与执行框架

面向动态负载，减少 GPU 内部的等待与数据搬运。
开发者描述计算与依赖，框架安排跨 SM 协作。

GPU Kernel 的负载日趋动态化

主要 LLM 算子面临
动态且不规则的负载

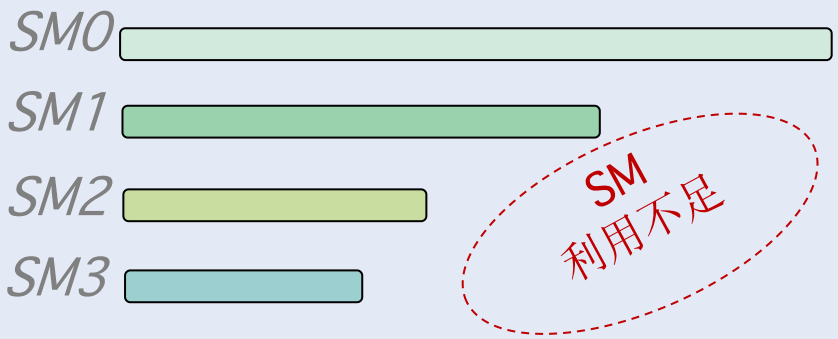
解码 Attention：batch 大小与序列长度动态变化



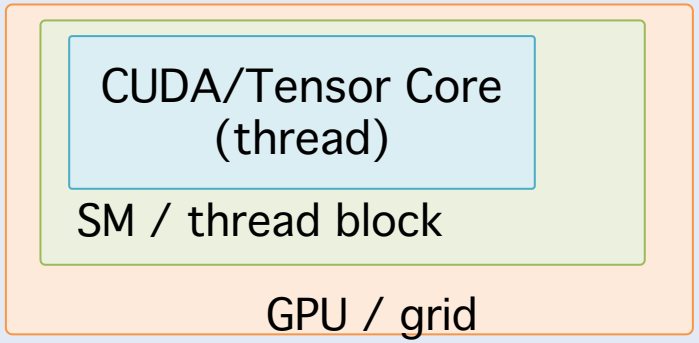
MoE：专家负载动态变化且分布不均

还包括：RMSNorm、NSA indexer、Sampling、
Cross-Entropy 与 ANNS cluster probing

静态映射导致 SM 利用率不足



例如：早期 FlashAttention

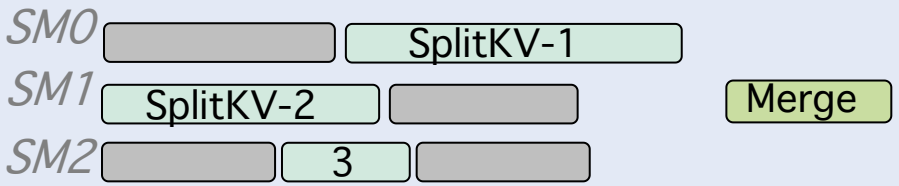


核心瓶颈：动态的跨 SM 依赖

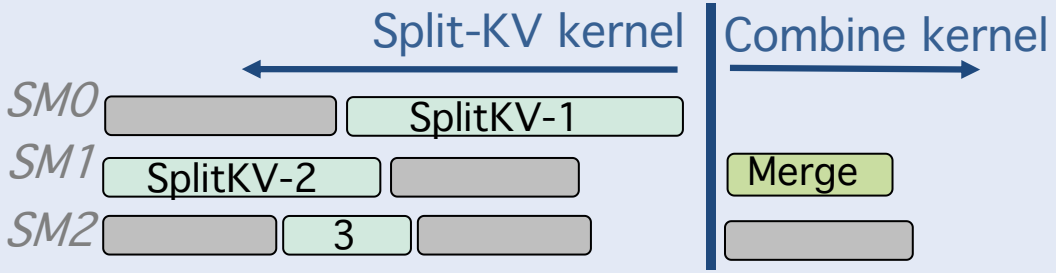
任务划分引入跨 SM 依赖

将大任务拆分到多个 SM。

后续阶段再合并各自的部分结果。

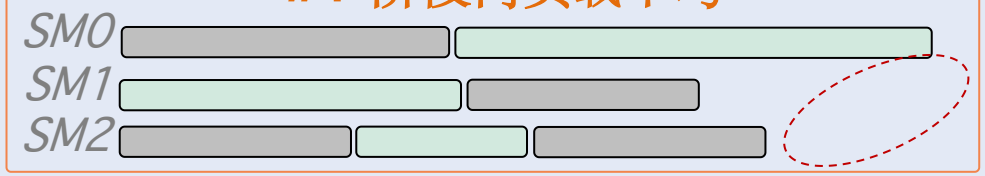


Kernel 边界强制阶段同步

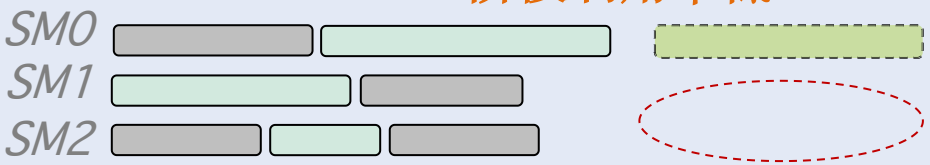


Kernel 屏障带来三类问题

#1 阶段内负载不均

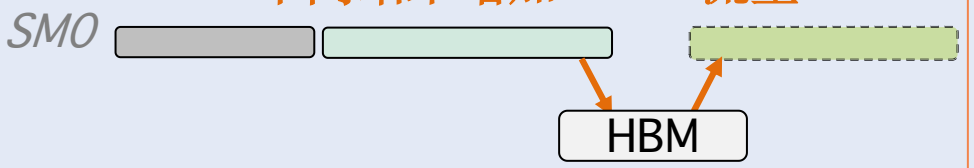


#2 Reduce 阶段利用率低



FlashMLA Combine：耗时 37%，SM 利用率 10%

#3 中间结果增加 HBM 流量

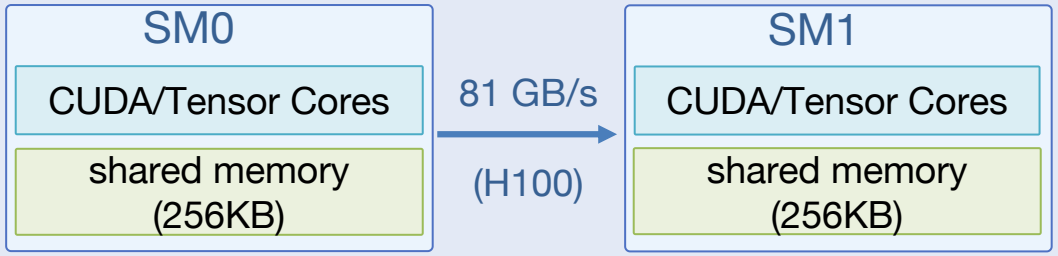


FlashInfer fusedMoE：占总流量的 20%

硬件机会：片上 SM 间互联

现代 GPU 支持 SM 之间直接互联

从 Hopper 开始，同一 **thread block cluster** 内的 SM 可互访 shared memory。



带宽提升 4 ×　　延迟降至 1/7

（相比经 HBM 中转的跨 SM 传输）

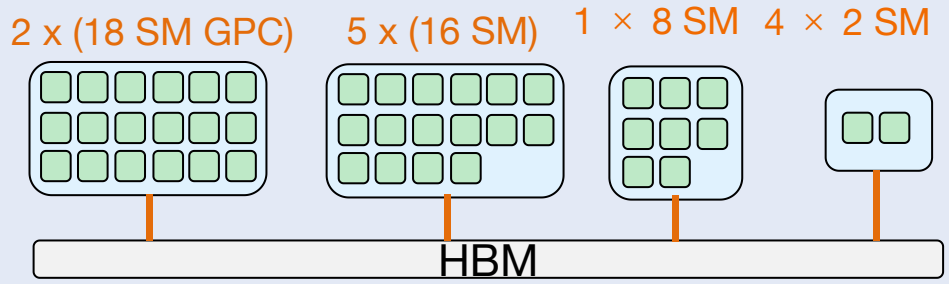
关键硬件能力：

Distributed Shared Memory	细粒度同步	异步传输

但互联范围局部化，且拓扑具有异构性

SM 互联限于 cluster 内，且大小不统一。

H100 拓扑



通信路径

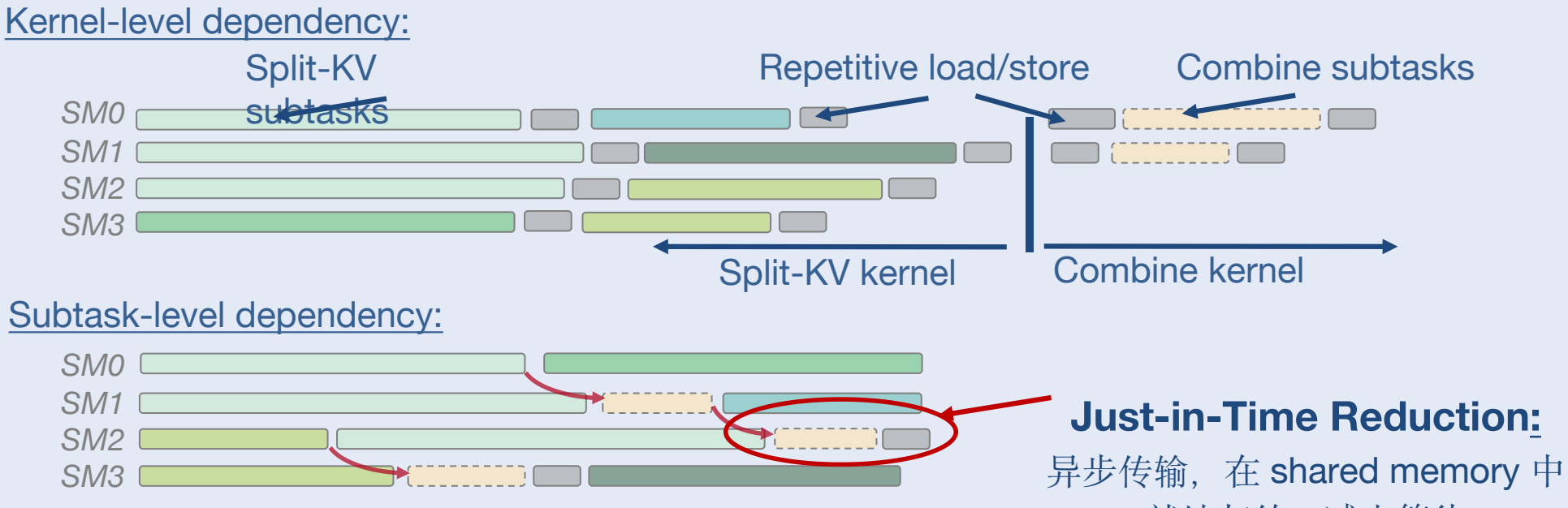
同一 cluster 内	片上 shared memory
跨 cluster	global memory

Meld：细粒度 SM 协作的执行流程

#1. 通过*细粒度 SM 协作拆解依赖*

将粗粒度 Kernel 级依赖拆解为子任务级依赖。

通过 SM-to-SM 传输与细粒度同步，实现计算、传输和归约重叠。

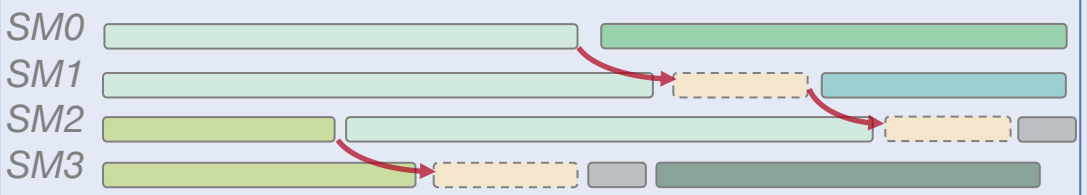


Meld：细粒度协作与动态映射

#1. 通过细粒度 SM 协作
拆解粗粒度依赖

将粗粒度 Kernel 级依赖拆解为细粒度子任务级依赖。

子任务级依赖：



Just-in-Time Reduction：

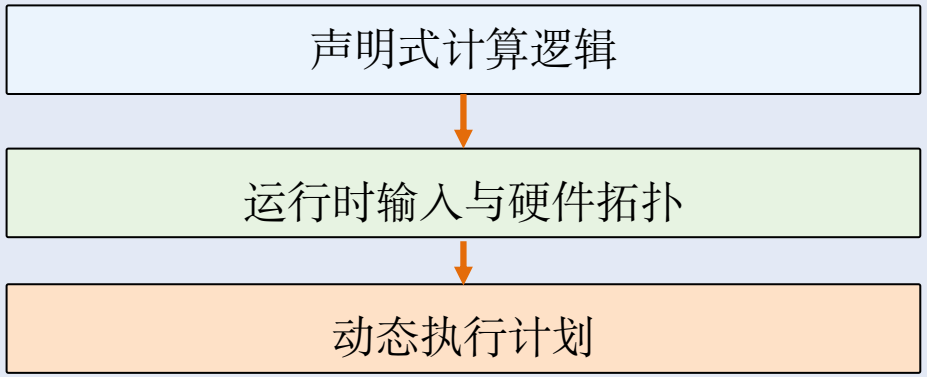
异步传输，在 shared memory 中

Just-in-Time Reduction

TMA 拷贝与 mbarrier 同步

#2. 通过解耦执行协调动态依赖

解耦计算逻辑与硬件映射，使执行适应动态输入。

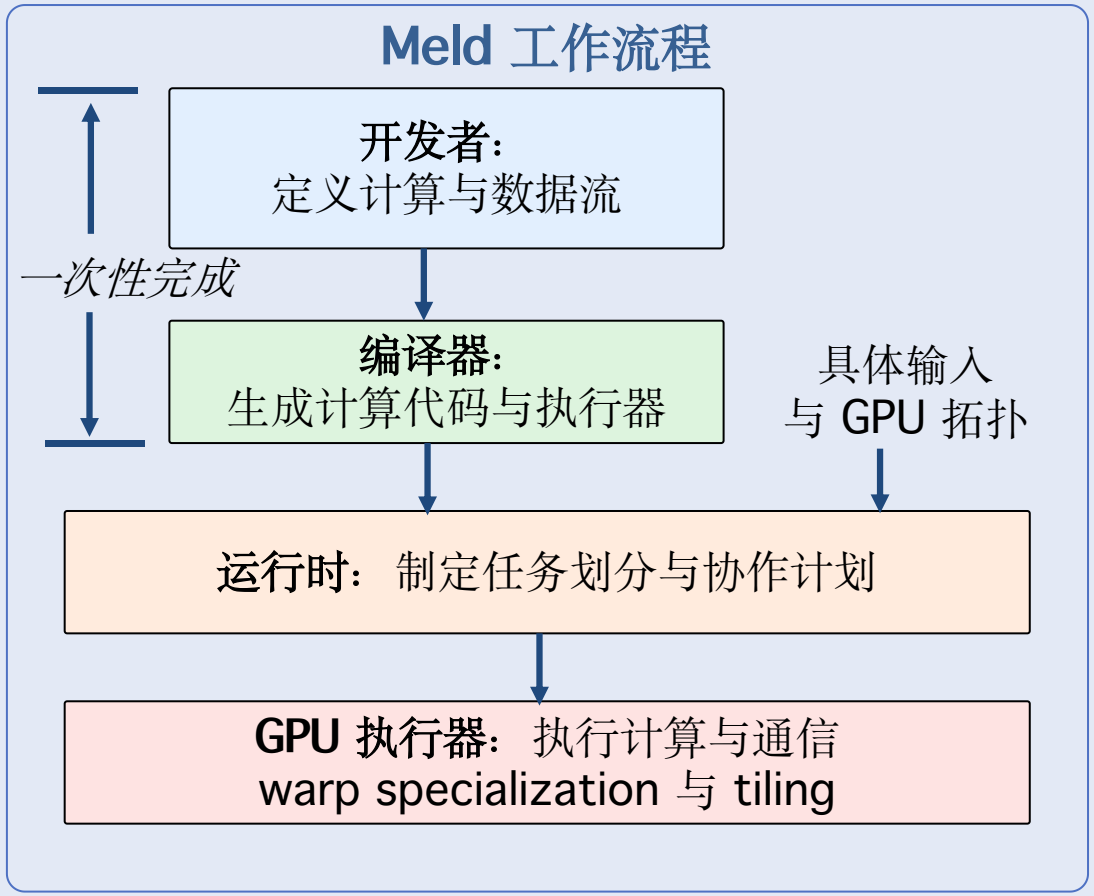


（任务划分、SM 映射、通信计划、tiling variant）

开发者定义计算；运行时决定如何执行、在哪里执行。

Meld：编译器、运行时与 GPU 执行器

将逐阶段执行多个 Kernel，转化为
动态协调的细粒度跨 SM 流水线。



各层协同

编写算子与数据流
将计算逻辑与硬件放置解耦。

依据局部性通信
cluster 内走片上互联，跨 cluster 走 HBM。

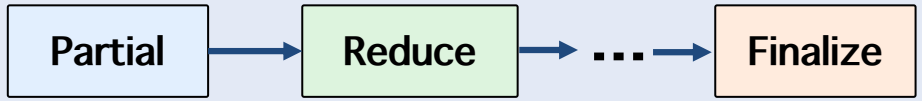
兼顾负载均衡与数据复用
划分任务，同时限制片外流量。

执行细粒度计划
保持高效的 SM 内部流水线。

Meld：通用数据流与共享输入复用

观察：算子多样，数据流模式相通

观察：多阶段算子共用 Partial/Reduce 模式。



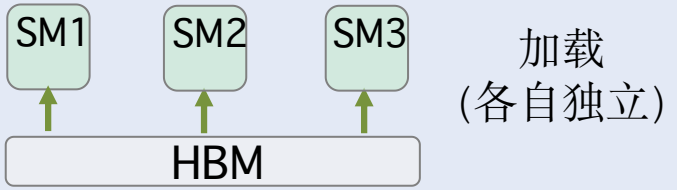
Meld：基于算子的声明式编程

将跨 SM 协作封装为一等算子。

Operator	Semantics
Iter/Map	Produce intermediates in parallel
Reduce	Merge the intermediates
Reshard/Gather	Redistribute/regroup interm.
Finalize	Materialize output

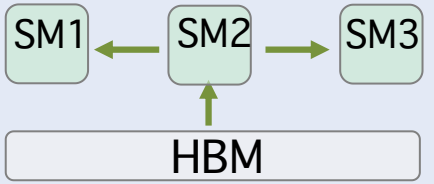
通过 Cooperative Loading 实现 Copy Absorption

重复加载相同输入，如 Attention 共享前缀 KV cache



Meld：Cooperative Loading

通过 TMA multicast 多播



编程语义

```
@Iter(..., affinity = shared_prefix_range)
def splitKV(...) -> Interm
    Copy(K[...], K_shared, absorbable=True)
```

Meld：兼顾负载均衡与数据局部性

跨 cluster：平衡计算与流量

将共享输入的任务分组。

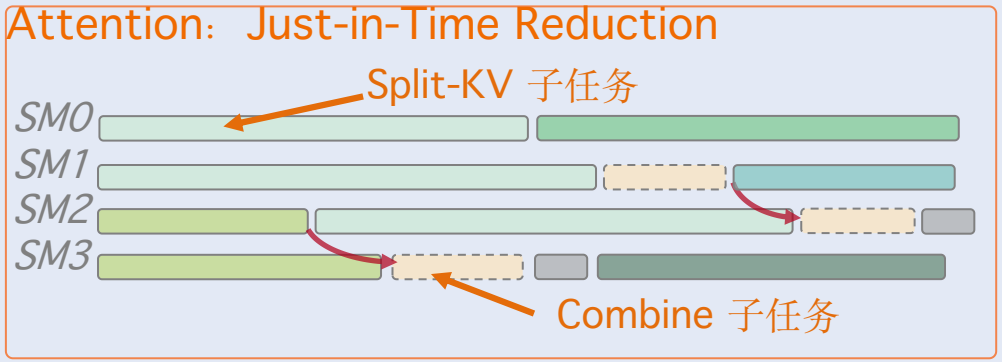
在硬件 cluster 间划分任务，
同时限制片外数据传输。

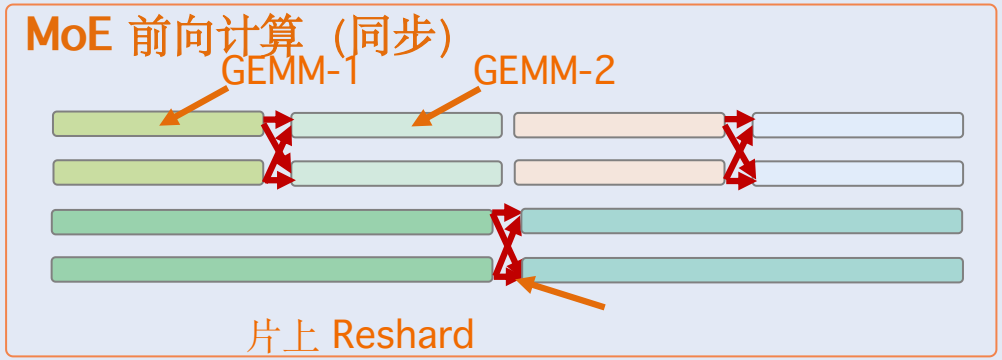
cluster 内：流水化处理依赖

输入就绪后，即调度细粒度子任务。
尽可能将中间数据
保留在片上。

cluster 内调度：
用跨 SM 流水线融合各阶段

将逐阶段执行转化为流动的子任务级流水线。



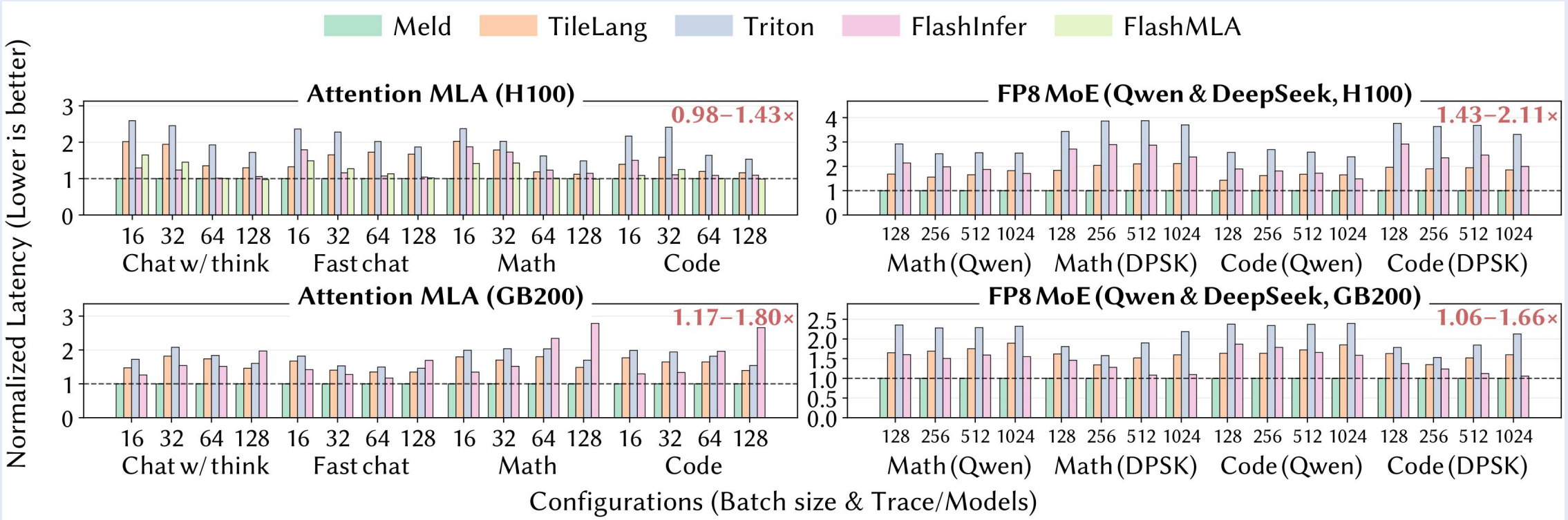


Meld：不同负载下的 GPU Kernel 性能

基于 TileLang 实现（>50K 行代码）；MLA 和 MoE 约 500–600 行 Python

GPU：H100、GB200、RTX 5090；覆盖六类算子

回放 Math、Code、Fast Chat 和 Chat-with-Thinking 轨迹

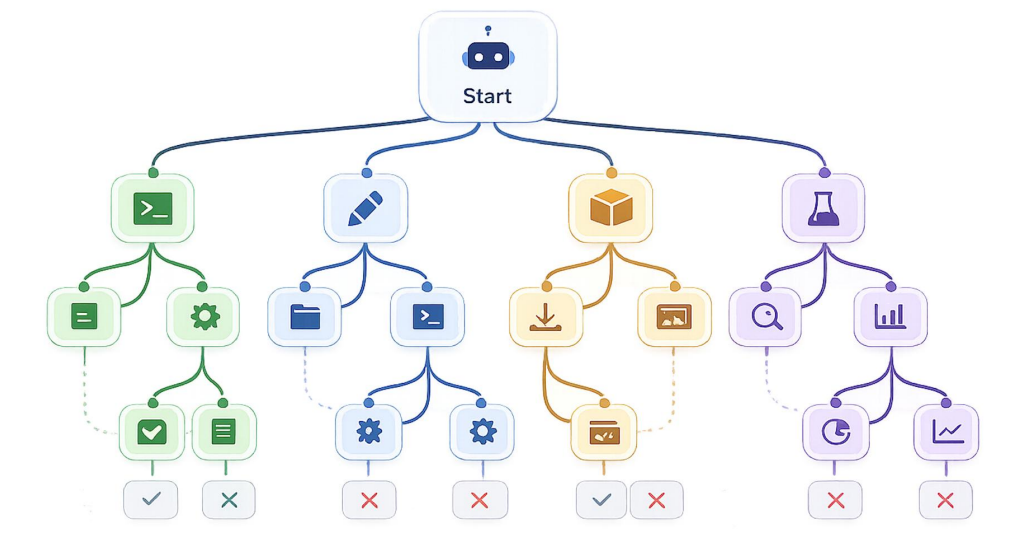


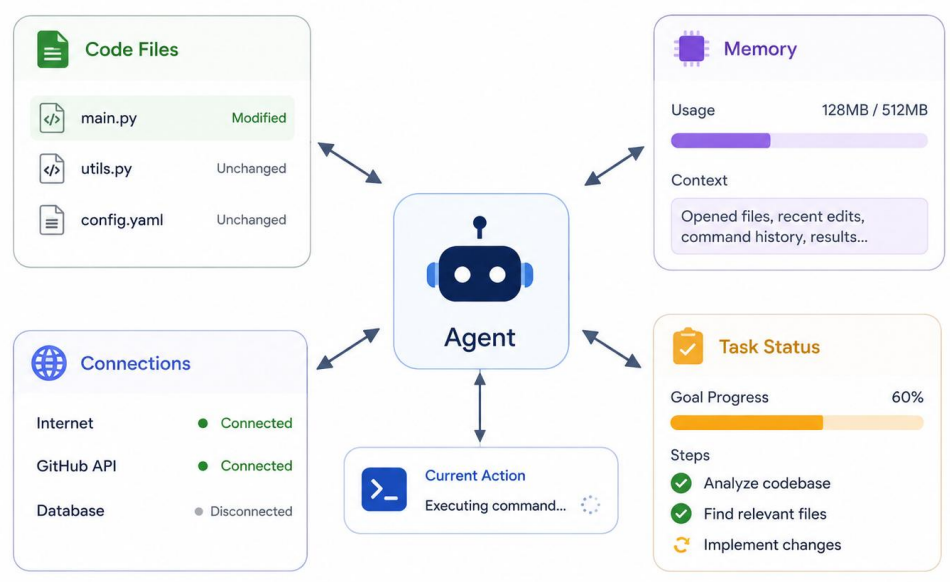
03

DeltaBox：Agent Sandbox 状态管理系统

面向 Agent 分支探索，降低环境切换开销。
快速保存、回滚和分叉文件与进程状态。

Agent 探索反复分叉与回滚环境状态



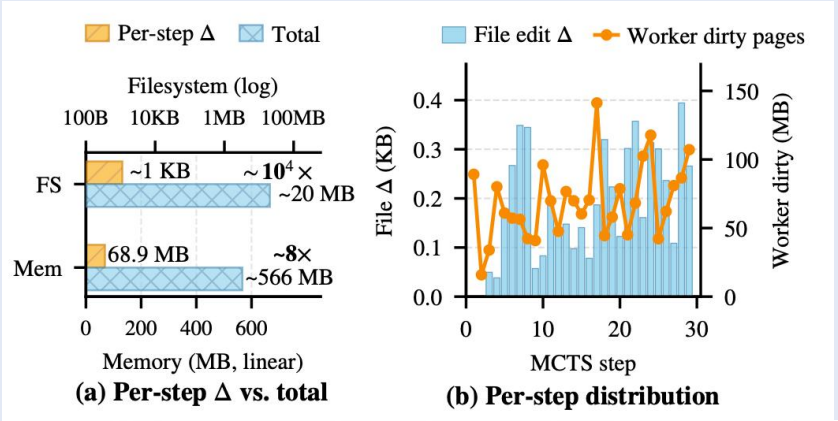


树搜索与 Best-of-N 需要反复分叉和回滚环境。

每个分支都可能修改文件、内存、进程和连接。

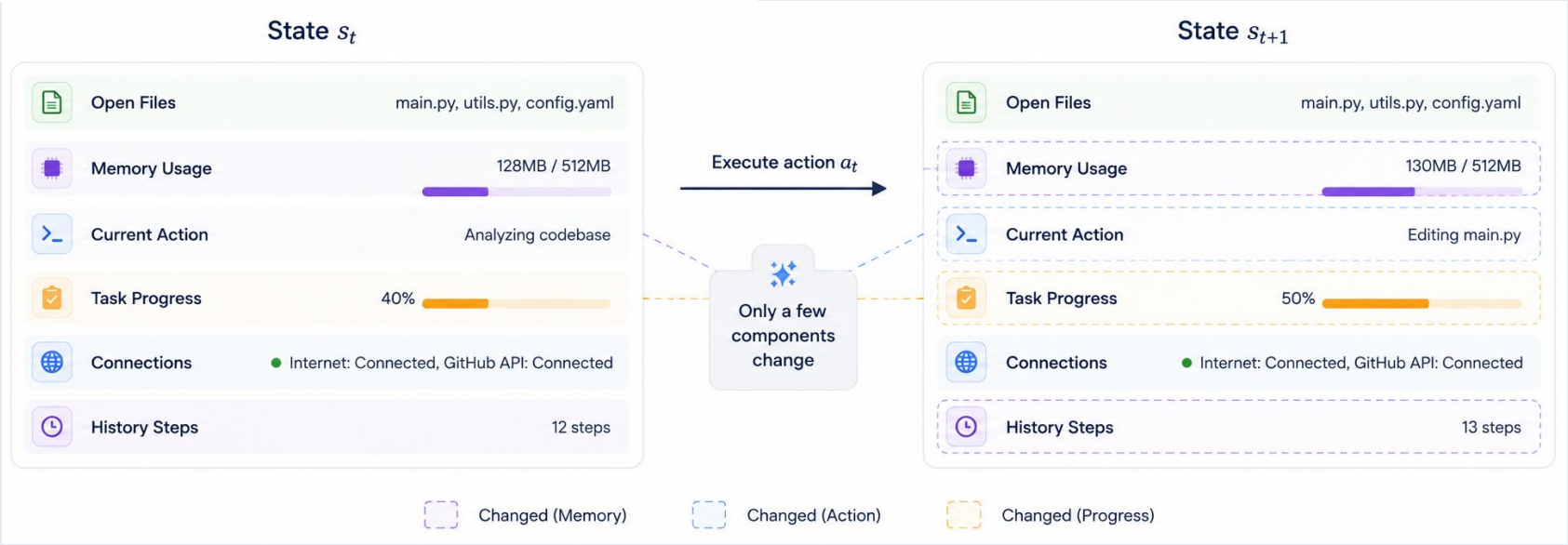
全量复制及容器/VM 快照通常耗时数百 ms 至数秒。

Agent 操作之间，大部分 Sandbox 状态可以复用

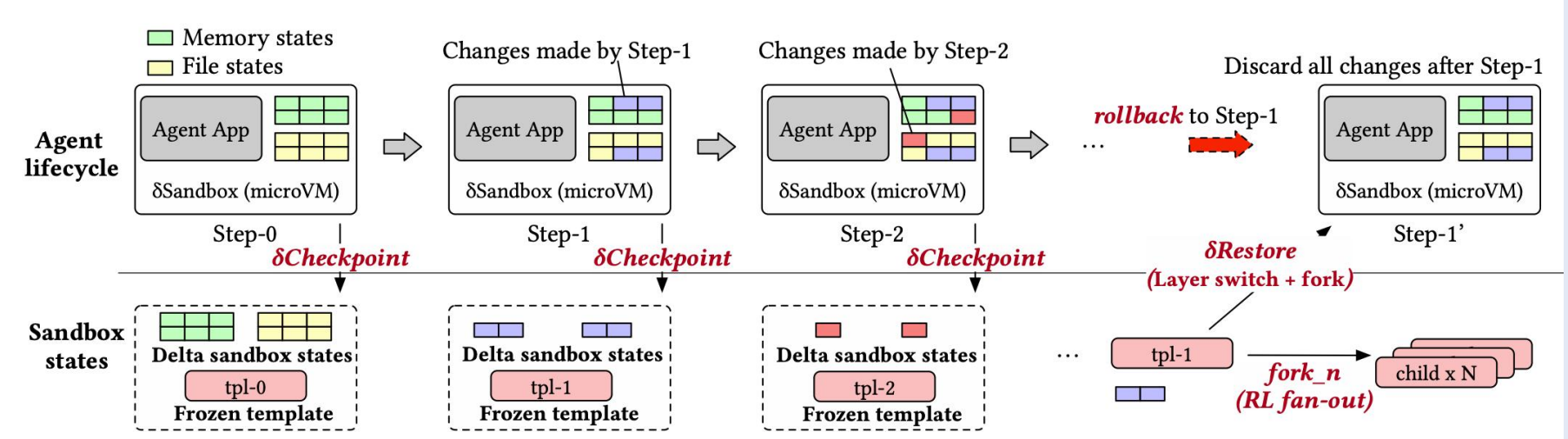


每次操作仅修改局部状态。

每步探索仅约 12% 的内存
和约 0.01% 的文件发生变化。



DeltaBox：毫秒级 Sandbox Rollback 与 fork

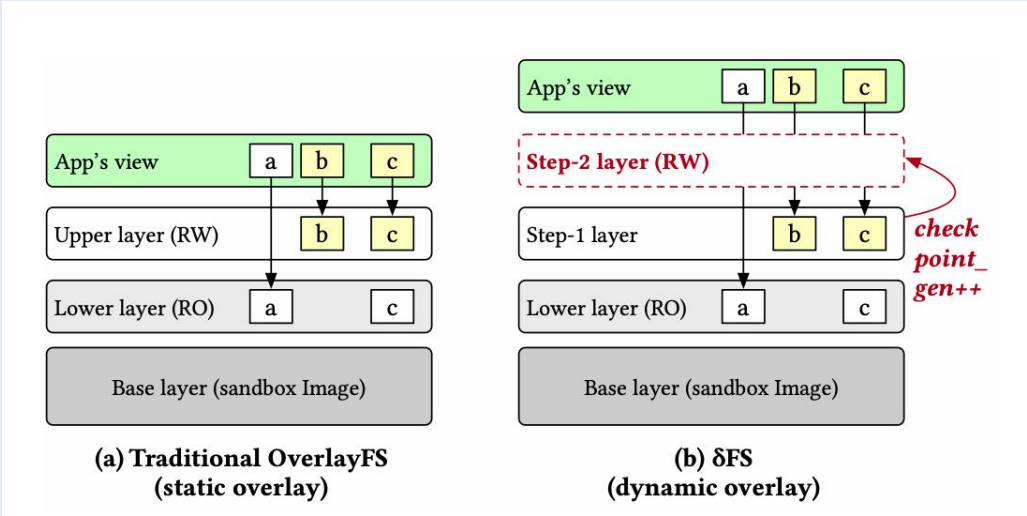


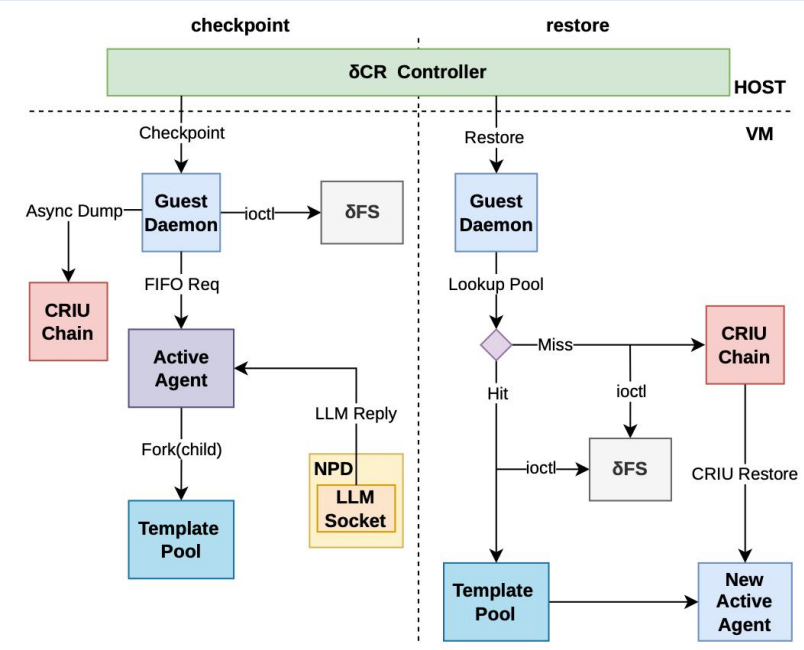
DeltaState：DeltaBox 中的检查点状态。

文件与进程状态保持一致，Checkpoint 只保存增量。

Dong et al. (2026). DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback.

DeltaBox： DeltaFS 与 DeltaCR 的分工

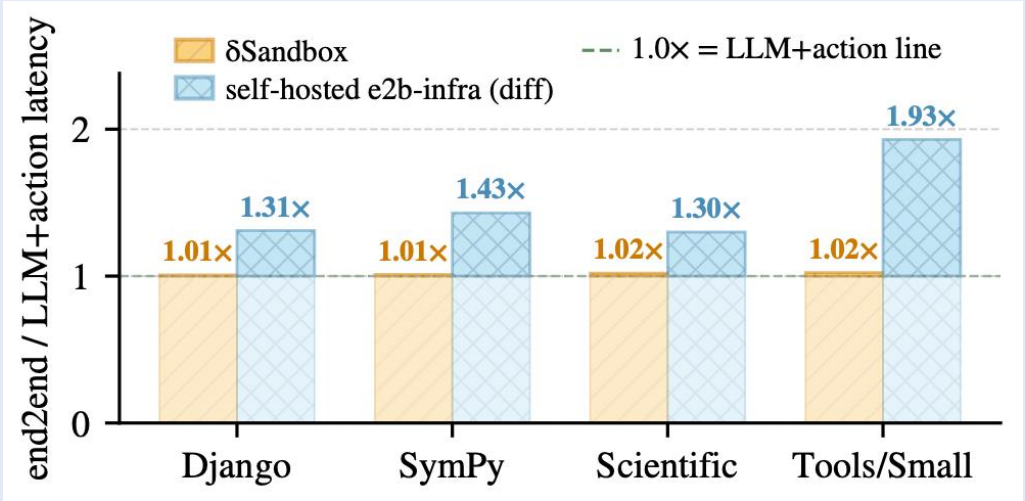




DeltaState 包含文件与进程状态，分别由 DeltaFS 和 DeltaCR 管理。

DeltaFS 通过 OverlayFS 层切换保存文件；DeltaCR 从冻结模板 fork 恢复进程。

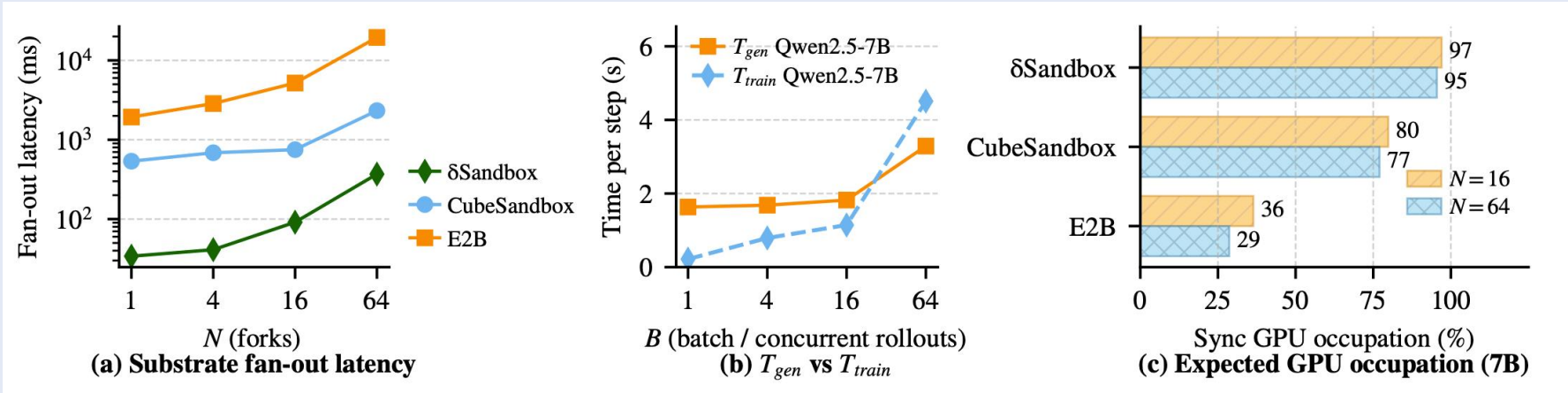
DeltaBox：以毫秒级切换加速 Agent 探索



Checkpoint 约 10 ms，Restore 约 2 ms

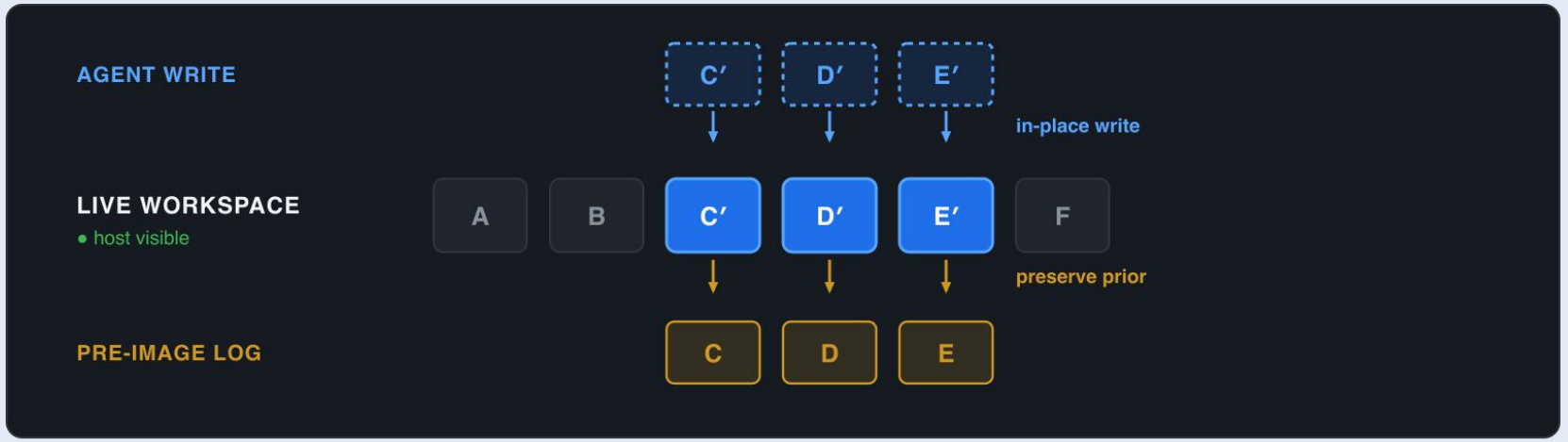
Agent 探索最高加速 1.9 ×

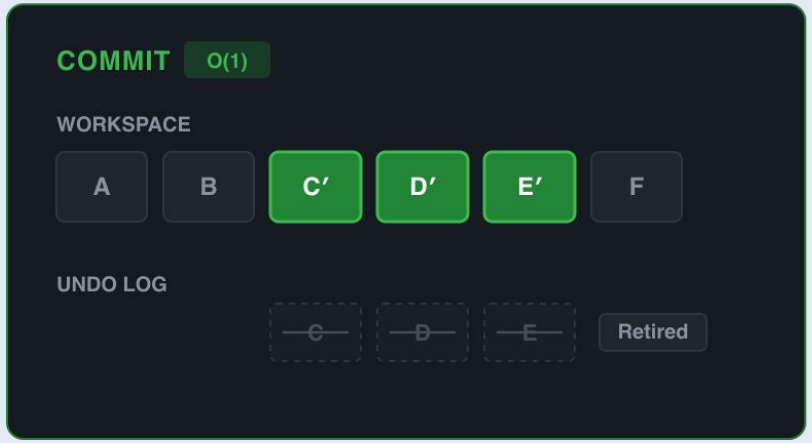
快速 fork 出 64 个环境

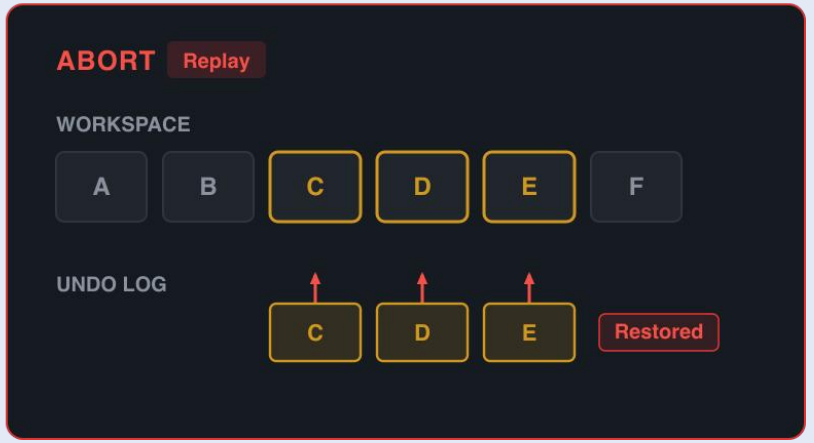


DeltaBox Lite：面向 Coding Agent 的审批与回滚，开箱即用

面向 AI Coding Agent 的轻量级文件系统运行时（Linux / macOS）







审查改动

Diff 展示文本差异，
标记二进制与大文件变更

Commit-Biased CoW

Commit 采纳改动
Abort 回放 Undo Log，
恢复受管路径

贴近日常开发

默认模式下，IDE 可
即时看到文件改动
支持 CLI、DSH、Pi 集成

总结：复用贯穿 RL 执行的三个层次

RhymeRL / 历史 token 与响应长度

投机式 Rollout 与历史感知调度
吞吐最高为 veRL 的 2.6×，保持一致的模型效果

Meld / 共享数据与 SM 协作

细粒度跨 SM 流水线与 Cooperative Loading
在 H100 与 GB200 上验证 MLA 和 MoE 性能

DeltaBox / 未变化的环境状态

增量 Checkpoint 与状态 Hot Switching
Checkpoint 约 10 ms，Restore 约 2 ms
Agent 探索最高加速 1.9×

QCon
全球软件开发大会

InfoQ 技术大会

Q&A

感谢聆听!

https://github.com/verl-project/verl-recipe/tree/main/specRL/histoSpec

https://github.com/He-Jingkai/tilelang.dataflow

https://github.com/delta-box

杜冬冬：Dd_nirvana@sjtu.edu.cn

和敬凯：hjk020101@sjtu.edu.cn



附录

备份材料

RhymeRL：机制、实验配置与详细结果

Meld：编程、通信与执行细节

历史相似性的成因

单步更新过大会引发灾难性遗忘

RL 采用多种方法避免剧烈更新

相邻 epoch 间的模型策略变化总体可控

更新幅度适中，

通常稳定在 10^{-1} 量级

Gradient clipping

归一化

$$\nabla_\theta L \propto \underbrace{\hat{A}_t}_{\text{Advantage}} \cdot \underbrace{r_t(\theta)}_{\text{Importance weight}} \cdot \underbrace{\nabla_\theta \log \pi_\theta(a_t|s_t)}_{\text{Score Function}} \quad (1)$$

PPO clipping

模型更新幅度

HistoSpec：基于树的历史 Rollout 管理

草稿生成速度决定投机解码的收益

→ RL 中每个 epoch 生成多条长响应

→ 每个 prompt 累计数十万 token

→ 传统草稿方法（如 N-gram）难以高效运行

方案：离线分布式索引 + 在线本地投机

→ RL 通常间隔多个训练步后才再次采样同一请求

→ 将检索计算开销前移至索引阶段

+ History worker

→ 异步索引历史 Rollout 序列

→ 异步调度 prompt 至 Rollout worker 时传输索引缓存

→ 集群级分布式部署，利用空闲 CPU 资源

HistoSpec：基于树的历史 Rollout 管理

优势感知的路径选择

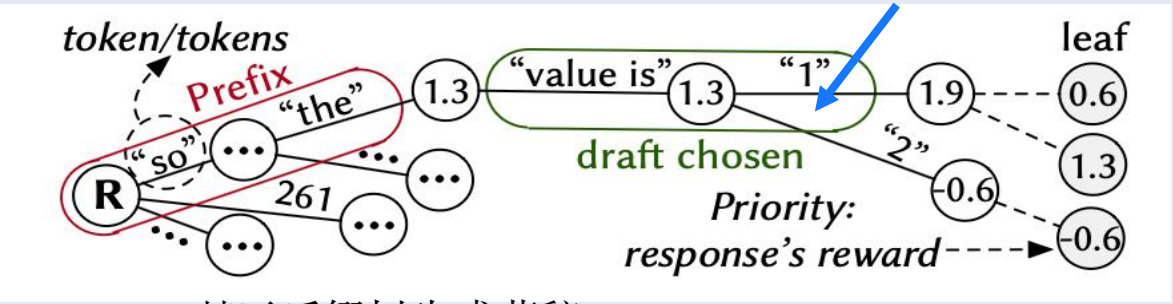
→ RL 为每个 prompt 生成多条响应，探索多条解题路径

→ 同一前缀可分叉为不同的后缀分支

→ 观察：RL 将模型优化方向推向高奖励解

→ 优势感知方案：每个树节点设置优先级，
按其分支的奖励之和加权

选择高奖励后缀分支



基于后缀树生成草稿

HistoPipe：分布感知的混合流水线

RL 系统中的现有流水线

→ 全同步

→ Rollout 长尾造成流水线气泡

→ Rollout 与训练共置的开销

→ One-step-off

→ 使用落后一次更新的模型权重执行 Rollout

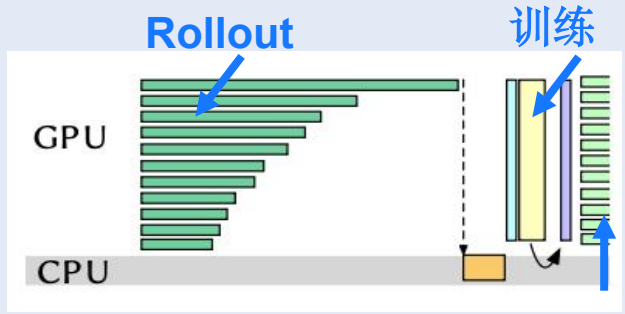
→ 仍存在 Rollout 不均衡气泡

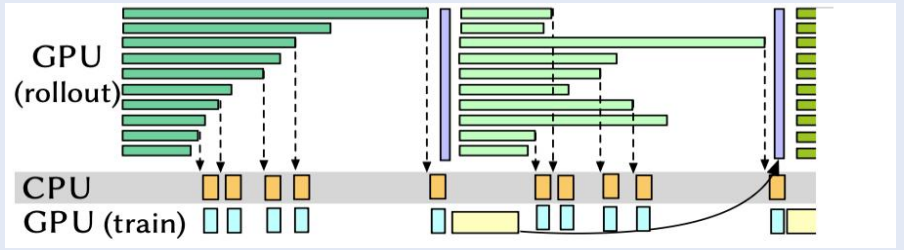
→ 全异步

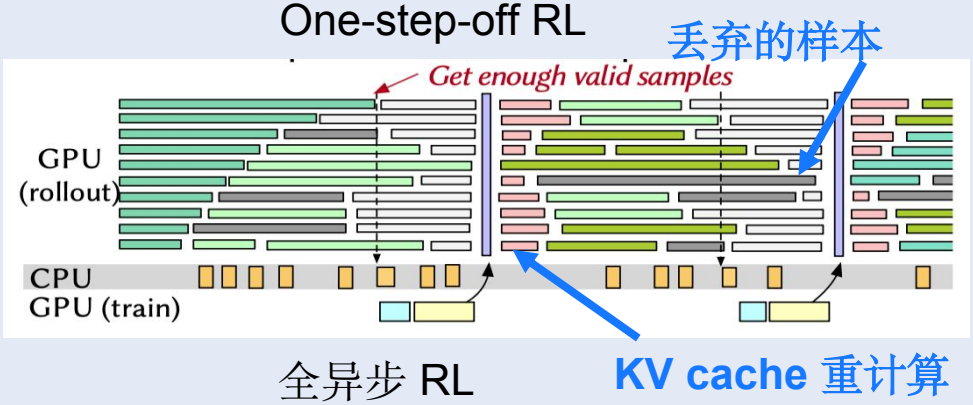
→ KV cache 重计算

→ 丢弃部分样本

→ 改变 RL 范式







HistoPipe： Two-tier Scheduling

挑战： Rollout 长度呈长尾分布

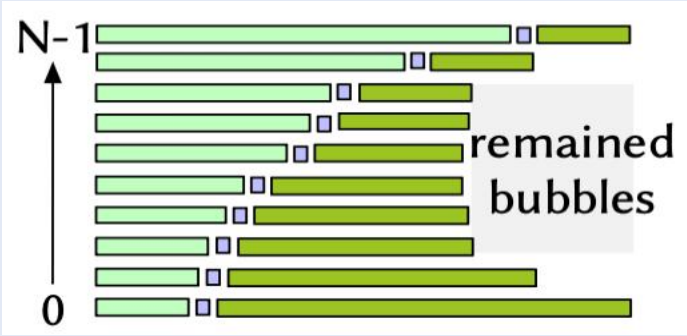
→ 连续训练步之间难以实现完全互补的负载

第二层调度： 通过资源分配重塑分布

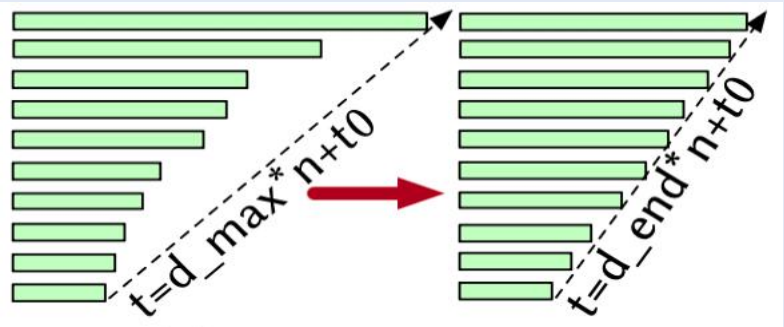
→ 为短响应组和中等响应组分配较少 GPU

→ 为少数长响应组分配更多 GPU

→ 建模为整数非线性规划问题



朴素设计中的流水线气泡



第二层调度

实验配置

硬件

→ 8-16 个节点，64-128 张 Hopper GPU（80 GB）

→ 每节点：8 张 GPU，2 颗 Intel Xeon Sapphire Rapids CPU（96 核），1.9 TB 主机 DRAM，9 * 400 Gb/s ConnectX7 InfiniBand NIC

模型

→ 主要模型：Qwen3-8B-Base, Qwen3-14B-Base, Qwen2.5-32B

→ 泛化验证：Llama3.1-8B, DeepSeek-R1-Distill-Qwen-7B

算法

→ GRPO & DAPO

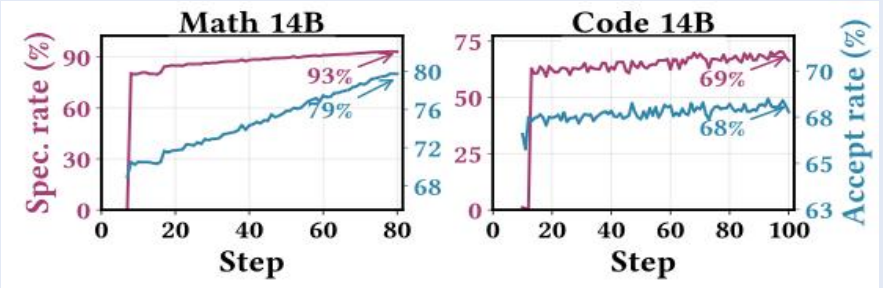
→ 主要参数：temperature = 1.0, top_p = 1.0, top_k = -1, min_p = 0.0

HistoSpec

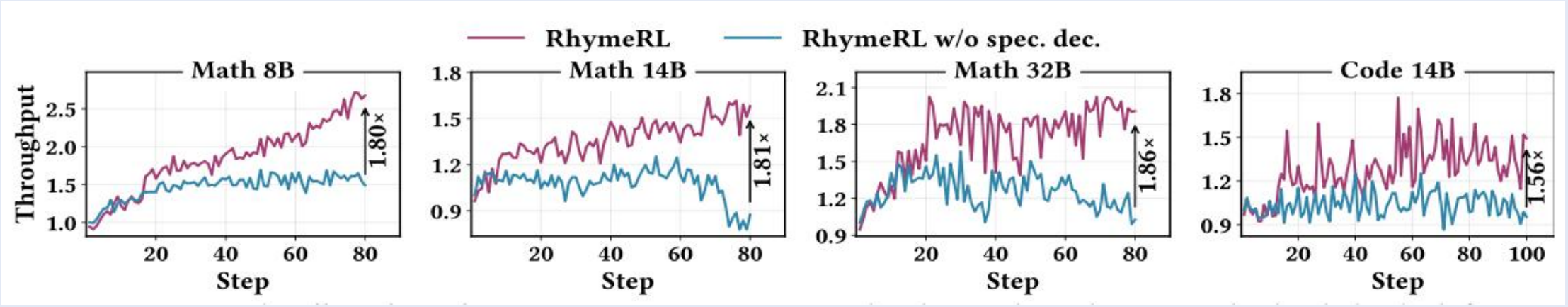
→ 单步 Rollout 吞吐最高达基线的 **1.86x**

→ 收益随训练过程逐步提升

→ 接收率为 **65%-79%**



投机率与接收率



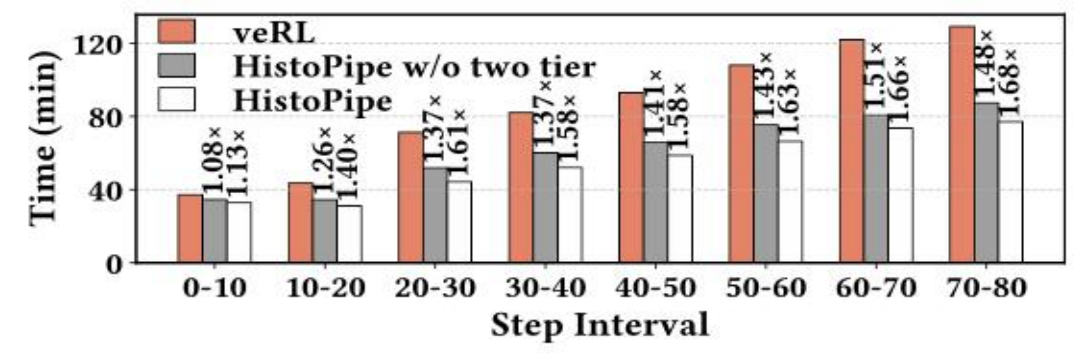
HistoSpec 各训练步的 Rollout 吞吐提升

HistoPipe

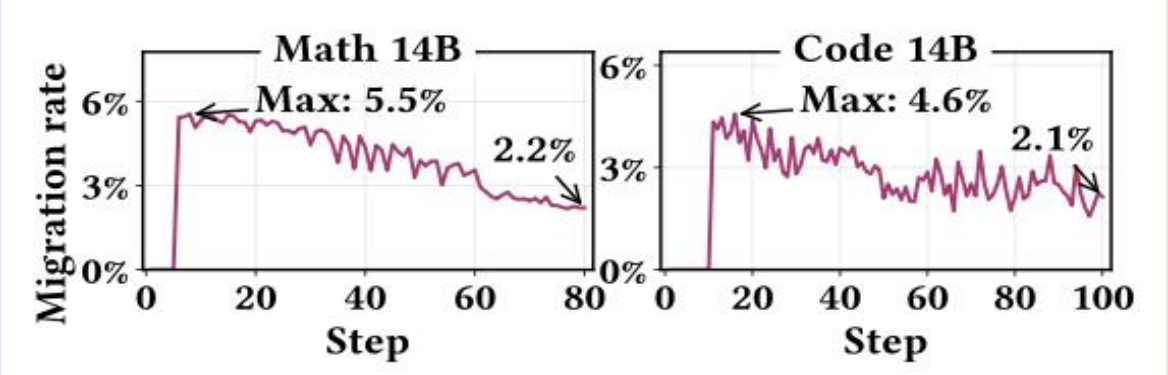
→ HistoPipe 对每 10 步训练的加速比最高达 **1.68x**

→ Two-tier Scheduling 相对朴素混合流水线最高加速 **1.14x**

→ 迁移样本占 **1.6%-5.5%**



HistoPipe 在各训练步的收益

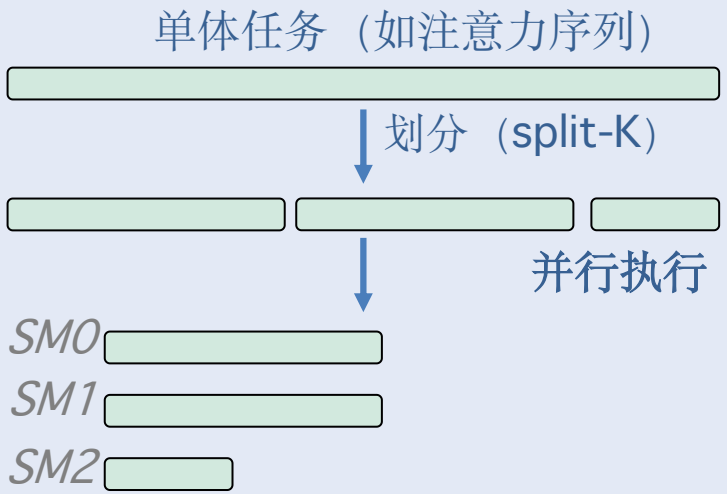


迁移样本比例

增加并行度，也会引入多个执行阶段

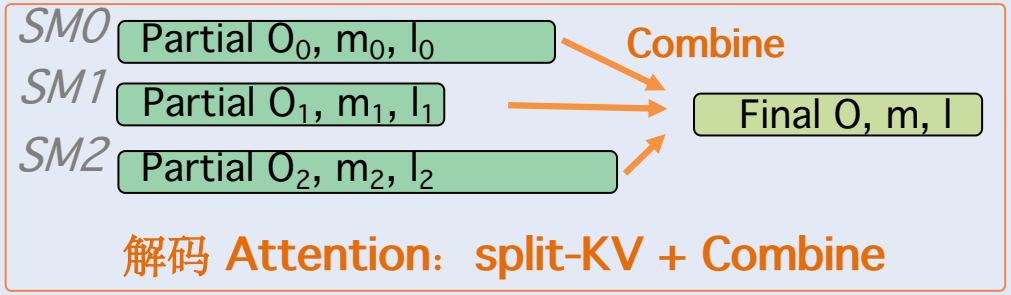
任务划分释放更多并行度

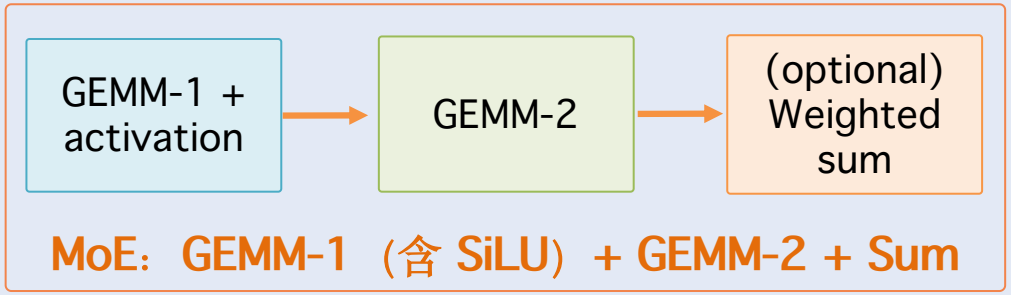
将大任务拆分至多个 SM，缓解长尾。



但计算也被拆为多个阶段

并行子任务产生的中间结果，需要在后续阶段合并。



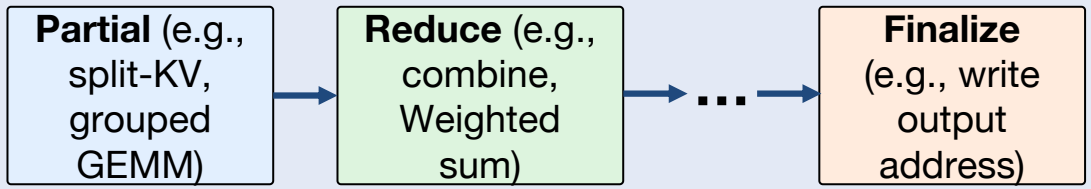


关注重点：执行阶段之间动态的跨 SM 依赖

设计 1：面向 SM 协作的声明式编程

观察：算子多样，数据流模式共通

观察： 多阶段算子具有共通的 Partial/Reduce 执行模式。



GPU Kernels	Pattern
Attention	Partial → Reduce → Finalize
MoE Forward	Partial → Partial → Reduce → Finalize
NSA Indexer	Partial → Reduce → Finalize
RMSNorm	Partial → Reduce → Partial → Finalize
Cross-Entropy	Partial → Reduce → Finalize

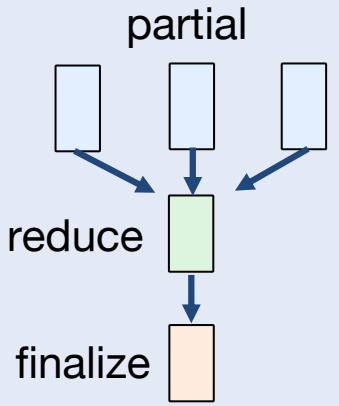
Meld：基于算子的声明式编程

将跨 SM 协作表达为一等算子。

```
@Iter
def splitKV(...) -> Interm

@Reduce
def combine (list[Interm])-> Interm

@Finalize
def finalize(Interm)
```



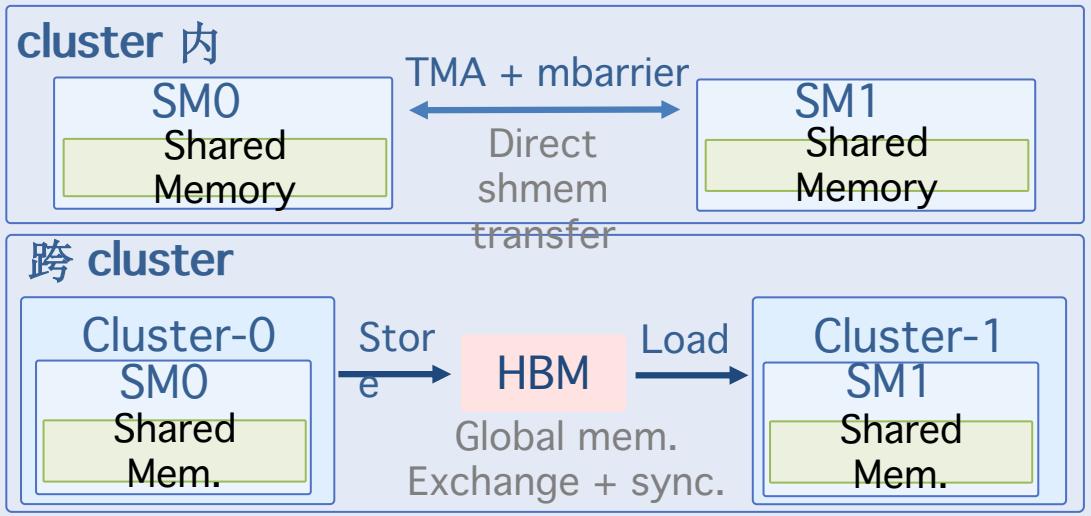
例如：解码 Attention

Operator	Semantics
Iter/Map	Produce intermediates in parallel
Reduce	Merge the intermediates (tree/cascaded/…)
Reshard/Gather	Redistribute/regroup interm.
Finalize	Materialize output

设计 2：适配局部且异构的互连

混合通信原语

cluster 内利用高速片上互连，跨 cluster 访问自动回退至 HBM。



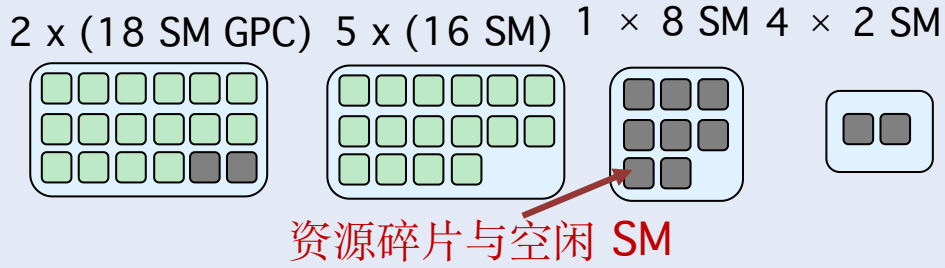
Primitive	Intra-Cluster	Inter-Cluster
send/recv	TMA SM-to-SM	HBM + polling
broadcast	Multicast TMA load	/
allGather/allReduce	TMA SM-to-SM	Two phase

异构感知的 cluster 级派发

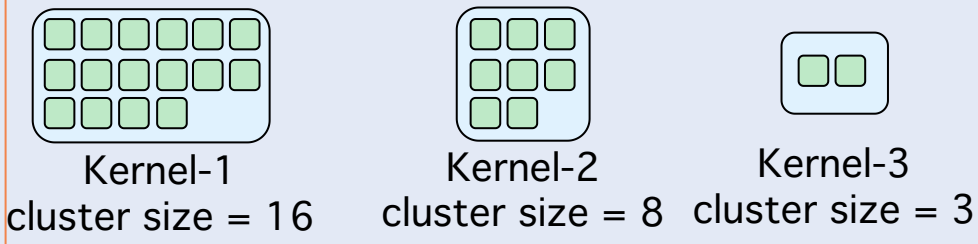
cluster 大小不一，

打破 CUDA 的统一 cluster 抽象。

统一 CUDA 抽象下的异构 cluster：



异构感知的混合 cluster 启动：



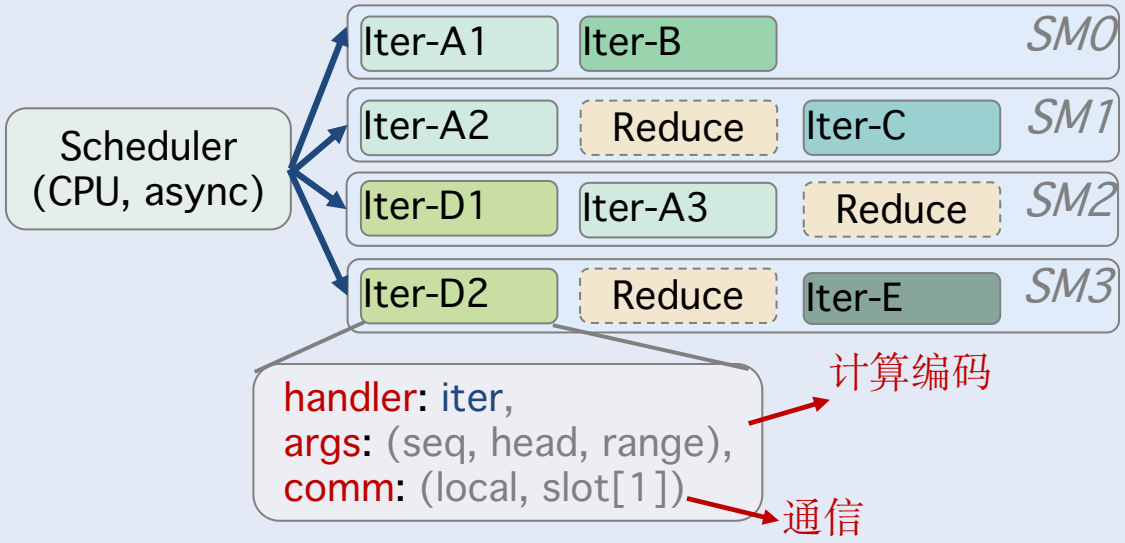
互不冲突的配置，并发执行

设计 4：高效执行动态计划

各 SM 独立的指令队列
实现精确映射

通过各 SM 的指令队列，将执行计划精确映射到指定 SM。

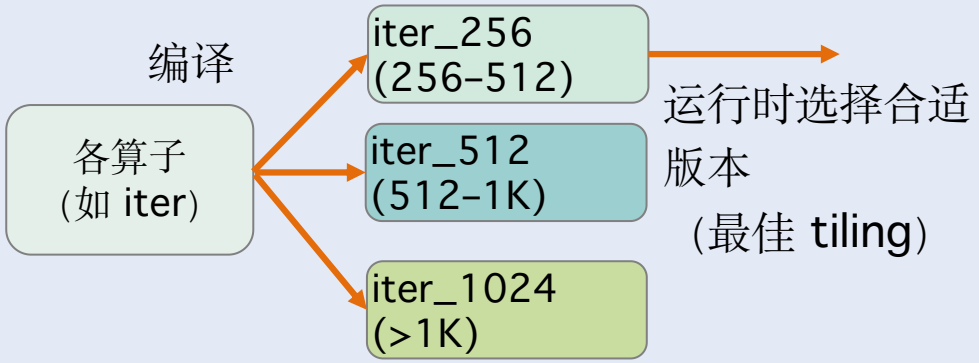
由软件管理各 SM 的指令队列



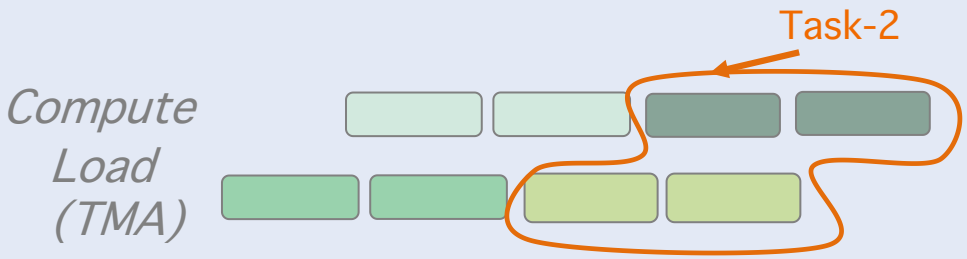
- 绕过黑盒调度器
- 按硬件拓扑放置任务

保持高效执行
适配细粒度任务

自适应选择 handler



跨 handler 的 SM 内流水线融合



利用 warp specialization 实现 SM 内融合

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师